

Manual Técnico

Arquitectura, modelo de datos, backend, frontend, calidad, integración continua y puesta en marcha de **PokéDex Manager**.

FastAPI · SQLAlchemy · PostgreSQL 18
Next.js 16 · React 19 · TanStack Query
Model Context Protocol · Turborepo

PROYECTO	PokéDex Manager
VERSIÓN	1.0
FECHA	13 de agosto de 2026
ESTADO	Vigente



Control del documento

Campo	Valor
Título	Manual Técnico — PokéDex Manager
Versión	1.0
Fecha	13 de agosto de 2026
Estado	Vigente
Clasificación	Uso interno
Repositorio	https://github.com/Skulltulaidm/pokedex-manager (público)
Rama documentada	main
Revisión del código	c5d3bd3
Audiencia	Ingeniería de software, QA, operaciones
Despliegue en producción	https://web-production-78a4b.up.railway.app
API en producción	https://api-production-81ae.up.railway.app
Documento hermano	Manual de usuario (mismo repositorio)

Historial de revisiones

Versión	Fecha	Cambio	Estado
1.0	2026-08-13	Primera emisión. Cubre arquitectura, modelo de datos, backend, frontend, requisitos, QA, CI/CD, instalación y bitácora de decisiones.	Vigente

Qué contiene y qué no

Alcance

Este documento describe **el sistema tal como está construido**, verificado leyendo el código de la revisión indicada arriba y ejecutando sus suites de pruebas. Las cifras que aparecen (número de rutas, de tablas, de pruebas, de herramientas MCP) se obtuvieron de esa revisión, no de una estimación.

Fuera de alcance: la operación día a día del producto para una persona usuaria —eso está en el manual de usuario— y cualquier funcionalidad planificada que todavía no exista en el repositorio.

Convenciones

- `ruta/al/archivo.py` es una ruta relativa a la raíz del repositorio.
- `identificador` es una función, clase, columna o variable de entorno tal cual aparece en el código.

- Las citas de código son literales; los comentarios que las acompañan también están en el repositorio.
- Los identificadores de requisito siguen **RF-*nn*** (funcional), **RNF-*nn*** (no funcional) y **HU-*nn*** (historia de usuario).

Índice general

1	Introducción	5
1.1	El problema	5
1.2	Qué hace el sistema	5
1.3	Glosario	6
2	Requisitos	7
2.1	Requisitos funcionales	7
2.2	Requisitos no funcionales	9
2.3	Historias de usuario	10
2.4	Trazabilidad	11
3	Arquitectura	12
3.1	Vista de contexto	12
3.2	Vista de contenedores	12
3.3	El monorepo	14
3.4	Tecnologías	15
3.5	Reglas de arquitectura	15
4	Modelo de datos	16
4.1	Dos esquemas, dos herramientas de migración	16
4.2	Catálogo y colección	17
4.2.1	Por qué <code>card_price</code> es una fila por carta y por día	17
4.2.2	Por qué un sobrante es <code>SUM(quantity) > 1</code>	17
4.2.3	Por qué el único de <code>collection_item</code> usa <code>NULLS NOT DISTINCT</code>	17
4.3	Intercambio, conversación y mensajería	17
4.3.1	Una oferta tiene destinatario; una publicación, no	17
4.3.2	Por qué la condición se copia y la cantidad no se guarda	17
4.3.3	Los multiplicadores de condición	22
4.3.4	Un hilo es un par ordenado	22
4.3.5	El historial del asistente se guarda íntegro	22
4.4	Índices y restricciones que importan	23
4.5	Migraciones	23
5	Backend	24
5.1	Capas	24
5.2	<code>CommittingRoute</code> : confirmar antes de responder	24
5.3	Paginación tipada: <code>Page[T]</code>	25
5.4	Autenticación	25
5.5	Catálogo de endpoints	26
5.6	La resolución de un escaneo	28
5.7	La lectura diaria de precios	28
5.8	El servidor MCP	29
5.9	El agente	29
5.10	Almacenamiento de imágenes	30

5.11	Configuración	31
6	Frontend	32
6.1	Estructura de rutas	32
6.2	Autenticación en el cliente	32
6.3	El envoltorio del cliente generado	33
6.4	Regeneración del cliente tipado	33
6.5	TanStack Query	34
6.6	La URL como estado	34
6.7	El sistema de diseño	34
6.8	Empaquetado	35
7	Flujos	36
7.1	Escaneo de una carta	36
7.2	Un turno del asistente sobre MCP	36
7.3	Tomar una publicación del tablón	36
8	Calidad	41
8.1	Estrategia	41
8.2	Las pruebas de la API	41
8.3	Las pruebas de la web	42
8.4	Análisis estático	43
8.5	Ejecución de referencia	44
9	Integración continua y despliegue	45
9.1	El pipeline	45
9.1.1	Job <i>API</i>	45
9.1.2	Job <i>Web</i>	45
9.2	Imágenes	47
9.3	Forma del despliegue	47
10	Manual de instalación	49
10.1	Requisitos previos	49
10.2	Puesta en marcha, paso a paso	49
10.2.1	1 · Clonar y preparar el entorno	49
10.2.2	2 · Levantar la base de datos	49
10.2.3	3 · Instalar dependencias de JavaScript	50
10.2.4	4 · Migrar el esquema <i>auth</i>	50
10.2.5	5 · Migrar el esquema <i>pokedex</i> y sembrar el catálogo	50
10.2.6	6 · Arrancar	50
10.2.7	7 · El modelo (opcional)	50
10.3	Verificación de la instalación	51
10.4	Datos de ejemplo	51
10.5	Todo en contenedores	51
10.6	Problemas frecuentes	52
11	Bitácora de decisiones	53
A	Anexos	56
A.1	Variables de entorno, completas	56
A.2	Comandos de referencia	57
A.3	Conectar un asistente externo por MCP	57
A.4	Cómo se construye este documento	57

1

Introducción

1.1 El problema

Guardar cartas es fácil; leerlas no. Una caja de zapatos con mil cartas de Pokémon no responde ninguna de las tres preguntas que su dueño se hace: qué tengo, cuánto vale y qué me falta. El almacenamiento no es la parte interesante del problema —una hoja de cálculo almacena— sino convertir un montón en algo consultable, valorable e intercambiable.

PokéDex Manager hace eso, y expone los mismos datos por dos caminos: una API REST que consume la aplicación web, y un **servidor MCP** al que puede conectarse cualquier asistente. El chat de la propia aplicación es cliente de ese servidor MCP, no una segunda ruta hacia los datos. Esa decisión se explica en el capítulo 5 y se ve en los flujos del capítulo 7.

1.2 Qué hace el sistema

Capacidad	Qué significa técnicamente
Escaneo	Una foto se normaliza, se guarda, se transcribe con un modelo de visión y se resuelve contra el catálogo puntuando cada señal. El usuario confirma.
Catálogo	card_set , card y species espejados de TCGdex y PokeAPI. La carta es una impresión; la especie es el Pokémon que retrata.
Colección	Grupos homogéneos de copias con condición, idioma, gradeo y coste unitario.
Portafolio	Coste contra valor de mercado, posición por posición, con concentración y serie temporal a partir de una lectura de precio diaria.
Intercambio	Emparejamiento entre coleccionistas, ofertas, contraofertas y un tablón abierto donde una propuesta se publica a nadie en particular.
Mensajería	Hilos directos entre dos coleccionistas, con estado de lectura.
Asistente	Agente sobre 13 herramientas MCP, con memoria de hechos duraderos, respondiendo en español y en streaming.
Compartir	Enlace público revocable con una proyección distinta a la del dueño, más exportación CSV y JSON.

1.3 Glosario

Término	Definición en este sistema
Carta (card)	Una <i>impresión</i> concreta: identificador del origen, set, número, rareza, variantes e imagen.
Especie (species)	El Pokémon retratado. Se une a la carta por el dexId del origen, nunca por coincidencia de nombres.
Sobrante (<i>spare</i>)	Copia excedente de una carta: SUM(quantity) > 1 agrupado por usuario y carta. La primera copia es la colección; lo que sobra es inventario.
Posición	Todas las copias de una misma carta de un usuario colapsadas en una fila, con coste, valor y participación en el portafolio.
Oferta (trade_offer)	Propuesta <i>con destinatario</i> . Aceptarla registra el acuerdo; no mueve cartas.
Publicación (trade_listing)	Propuesta <i>sin destinatario</i> . Quien pueda cumplirla la toma, y tomarla es el acuerdo.
MCP	<i>Model Context Protocol</i> . El protocolo por el que un asistente externo lee la colección.
JWKS	Conjunto de claves públicas que publica Better Auth y con el que la API verifica los JWT sin llamar de vuelta.
Kubb	Generador que convierte el OpenAPI de la API en tipos, clientes y hooks de React Query.

2

Requisitos

Los requisitos de este capítulo están **derivados del código**, no redactados antes de él: cada uno corresponde a una ruta, un servicio o una restricción que existe en la revisión documentada. No se enuncia nada que el sistema no cumpla.

2.1 Requisitos funcionales

Tabla 2.1. Requisitos funcionales

ID	Requisito	Realización
RF-01	El sistema debe permitir crear una cuenta con correo y contraseña, iniciar y cerrar sesión.	Better Auth en <code>apps/web/lib/auth.ts</code> , montado en <code>/api/auth/*</code>
RF-02	El sistema debe emitir, a partir de la sesión, un JWT que la API acepte, y publicar las claves para verificarlo.	Plugin <code>jwt()</code> ; GET <code>/api/auth/token</code> y <code>/api/auth/jwks</code>
RF-03	El sistema debe aceptar la foto de una carta, normalizarla, transcribir lo impreso y proponer candidatos del catálogo con su puntuación y las señales que coincidieron.	POST <code>/api/v1/scans</code> ; <code>services/scan.py</code> , <code>services/resolve.py</code>
RF-04	El usuario debe poder confirmar qué carta era y en qué estado, y que eso ingrese a su colección.	POST <code>/api/v1/scans/{id}/confirm</code>
RF-05	El sistema debe permitir dar de alta una carta manualmente, fusionándola con el grupo idéntico si ya existe.	POST <code>/api/v1/collection</code> ; ON CONFLICT <code>uq_collection_item_group</code>
RF-06	El sistema debe listar la colección paginada, filtrable por tipo, generación y set, y ordenable por reciente, nombre, número o valor de posición.	GET <code>/api/v1/collection</code> con <code>CollectionFilters</code>
RF-07	El usuario debe poder editar y eliminar un grupo de copias.	PATCH y DELETE <code>/api/v1/collection/{id}</code>
RF-08	El sistema debe reportar composición por tipo y generación, cobertura por set y valor estimado, indicando cuántas cartas tienen precio.	GET <code>/api/v1/stats</code> ; <code>services/stats.py</code>
RF-09	El sistema debe permitir buscar en el catálogo publicado, abrir la ficha de una carta, ver su línea evolutiva y una reseña de la especie.	Router <code>catalog</code> ; <code>services/catalog.py</code> , <code>services/trivia.py</code>
RF-10	El sistema debe presentar la colección como portafolio: resumen, posiciones ordenables, concentración del valor y rendimiento contra coste.	<code>/catalog/market/*</code> ; <code>services/market.py</code>
RF-11	El sistema debe simular un intercambio sobre las tenencias y precios de hoy sin escribir ni reservar nada.	POST <code>/catalog/market/simulate</code>

Continúa en la página siguiente

Tabla 2.1. Requisitos funcionales (continuación)

ID	Requisito	Realización
RF-12	El sistema debe desglosar el mercado por set, ordenado por el valor de lo que falta.	GET /catalog/market/sets
RF-13	El usuario debe poder marcar cartas deseadas, y el asistente sugerirlas, distinguiendo el origen de cada entrada.	/wishlist; columna added_by
RF-14	El sistema debe señalar qué falta sólo en los sets ya empezados.	GET /api/v1/gaps; services/gaps.py
RF-15	El sistema debe emparejar coleccionistas cuando el intercambio funciona en ambas direcciones.	GET /api/v1/trades
RF-16	El sistema debe ofrecer un directorio de coleccionistas, su perfil y sus sobrantes.	/trades/collectors[/id[/spares]]
RF-17	El sistema debe permitir crear, responder, retirar y contraofertar propuestas de intercambio, validando cada lado contra los sobrantes reales.	/trades/offers*; services/trade.py
RF-18	El sistema debe permitir publicar un intercambio sin destinatario, listar el tablón indicando cuáles se pueden cumplir, tomar una publicación y cancelarla.	/trades/listings*
RF-19	Dos coleccionistas deben poder conversar en un hilo directo, con estado de leído.	/api/v1/messages*; services/direct.py
RF-20	El sistema debe informar qué ocurrió mientras el usuario no estaba, derivándolo de las filas existentes, y recordar hasta dónde leyó.	/api/v1/news; services/news.py
RF-21	El usuario debe poder preguntar en lenguaje natural sobre su colección y recibir la respuesta en streaming.	POST /api/v1/chat (SSE); agent/
RF-22	El asistente debe recordar hechos duraderos del usuario, y el usuario debe poder consultarlos y borrarlos.	/api/v1/preferences; herramientas remember / forget
RF-23	El usuario debe poder publicar su colección con un enlace revocable, y esa vista debe ocultar notas y fechas de adquisición.	/api/v1/share, GET /api/v1/public/{token}
RF-24	El sistema debe exportar la colección en CSV y JSON.	GET /api/v1/collection/export
RF-25	El sistema debe exponer la colección a asistentes externos por MCP, con la misma autenticación y la misma capa de servicio.	/mcp; mcp/server.py
RF-26	El sistema debe poder sembrar y actualizar el catálogo desde los orígenes externos.	pokedex-sync <set-id>...
RF-27	El sistema debe registrar una lectura de precio por carta y por día.	take_todays_prices() en el arranque; tabla card_price
RF-28	El sistema debe poder poblarse con un mercado de coleccionistas reproducible para desarrollo y demostración.	pokedex-seed, pokedex-demo

2.2 Requisitos no funcionales

Tabla 2.2. Requisitos no funcionales

ID	Atributo	Requisito y cómo se cumple
RNF-01	Seguridad	Los JWT se verifican con el algoritmo EdDSA <i>fijado en código</i> , no leído de la cabecera del token, contra JWKS y comprobando emisor, audiencia, expiración y sujeto no vacío. La API nunca llama de vuelta al servicio de sesión.
RNF-02	Aislamiento	Toda consulta de dominio se acota por user_id . Una herramienta MCP que no puede resolver a su usuario lanza UnauthenticatedError en lugar de recurrir a un valor por defecto.
RNF-03	Integridad	Las invariantes viven en la base: claves foráneas, CHECK quantity > 0 , CHECK de gradeo coherente, CHECK first_user_id < second_user_id , únicos y enums de Postgres.
RNF-04	Consistencia	Una mutación queda confirmada <i>antes</i> de que su respuesta salga, para que un refetch inmediato no pueda adelantarse al commit (CommittingRoute).
RNF-05	Idempotencia	Catálogo, precios y altas de colección se escriben con INSERT ... ON CONFLICT DO UPDATE . El sembrado deriva todas sus claves de una semilla, así que repetirlo no escribe nada nuevo.
RNF-06	Concurrencia	Tomar una publicación del tablón es un UPDATE ... WHERE status='open' ... RETURNING id : dos personas simultáneas producen un ganador y un 409.
RNF-07	Degradación	Sin GOOGLE_API_KEY el chat responde 503 y el escaneo degrada a entrada manual; ninguna otra función se ve afectada. CI corre en verde sin ninguna clave.
RNF-08	Tipado	mypy -strict sobre 87 archivos y TypeScript strict con noUncheckedIndexedAccess , ambos exigidos en CI.
RNF-09	Verificabilidad	Las pruebas de la API corren contra un Postgres real, cada una dentro de una transacción que siempre se revierte, así que enums, CHECK y ON CONFLICT se comportan como en producción.
RNF-10	Portabilidad	Cada imagen levanta el esquema que le pertenece antes de servir; la API espera al esquema auth porque sus claves foráneas apuntan ahí.
RNF-11	Límites de recurso	Subida máxima de 10 MB, imagen reescalada a 1600 px de lado largo, turno del agente acotado a 8 llamadas de herramienta y 10 peticiones, y un tope de paginación por endpoint.
RNF-12	Privacidad	La proyección pública es un esquema distinto al del dueño: notas y fechas de adquisición no salen de la cuenta. Un enlace revocado responde 404 igual que uno que nunca existió.
RNF-13	Rendimiento	Índice GIN de trigramas para la resolución por nombre, GIN sobre species.types , índices compuestos para leer un hilo o una conversación en orden, y subconsulta escalar correlacionada en las posiciones para no romper el joinedload .
RNF-14	Accesibilidad	La hoja de estilo anula animaciones y transiciones bajo prefers-reduced-motion .
RNF-15	Idioma	La interfaz y las respuestas del agente van en español; el código, los identificadores y los mensajes de commit, en inglés.

2.3 Historias de usuario

Tabla 2.3. Historias de usuario

ID	Historia	Criterio de aceptación
HU-01	Como coleccionista quiero fotografiar una carta para no teclear sus datos.	El sistema muestra candidatos ordenados y por qué coincide cada uno; si está seguro, propone uno solo. (RF-03)
HU-02	Como coleccionista quiero decidir yo qué se guarda, porque la foto puede ser ambigua.	Nada entra a la colección sin confirmación explícita. (RF-04)
HU-03	Como coleccionista quiero buscar y filtrar lo que tengo para encontrar una carta sin recorrer todo.	Filtro por tipo, generación y set, con orden y paginación estables. (RF-06)
HU-04	Como coleccionista quiero saber cuánto vale lo que tengo y cuánto me costó.	Valor de mercado, coste, ganancia y participación por posición, con el número de cartas sin precio a la vista. (RF-10)
HU-05	Como coleccionista quiero saber si mi valor depende de pocas cartas.	Concentración por tramos y cuántas cartas cargan la mitad del portafolio. (RF-10)
HU-06	Como coleccionista quiero probar un intercambio antes de proponerlo.	El simulador informa diferencia de valor y efecto sobre la concentración sin escribir nada. (RF-11)
HU-07	Como coleccionista quiero encontrar a alguien con quien el cambio funcione en ambos sentidos.	Sólo se listan emparejamientos con material en las dos direcciones. (RF-15)
HU-08	Como coleccionista quiero publicar un cambio sin dirigirlo a nadie.	La publicación queda en el tablón e indica a quién le cuadra; tomarla la cierra. (RF-18)
HU-09	Como coleccionista quiero negociar con palabras, no sólo con listas de cartas.	Hilo directo entre dos personas, con estado de leído. (RF-19)
HU-10	Como coleccionista quiero enterarme de lo que pasó mientras no estaba.	Feed derivado con las ofertas por responder, los tratos cerrados, los mensajes sin leer y los movimientos de precio de lo que quiero. (RF-20)
HU-11	Como coleccionista quiero preguntar en mi idioma en vez de aprender la interfaz.	El asistente responde en español, cita cartas enlazables y no inventa datos: sólo repite lo que le devolvieron las herramientas. (RF-21)
HU-12	Como coleccionista quiero enseñar mi colección sin dar acceso a mi cuenta.	Enlace público revocable, sin notas ni fechas de adquisición. (RF-23)
HU-13	Como usuario de un asistente propio quiero consultar mi colección desde él.	Servidor MCP con el mismo token y las mismas reglas; una sola herramienta escribe y no alcanza la colección. (RF-25)

2.4 Trazabilidad

Requisito	Módulo	Prueba
RF-03, RF-04	services/scan.py, services/resolve.py	test_scan_service.py (6), test_resolve_service.py (14)
RF-05 – RF-07	services/collection.py	test_collection_service.py (22)
RF-08	services/stats.py	test_stats_service.py (5)
RF-09	services/catalog.py, trivia.py	test_catalog_service.py (5), test_evolution.py (9), test_trivia_service.py (2)
RF-10 – RF-12	services/market.py	test_market_service.py (44)
RF-13, RF-14	services/wishlist.py, gaps.py	test_gaps_service.py (9)
RF-15 – RF-18	services/trade.py	test_trade_service.py (47)
RF-19	services/direct.py	test_direct_service.py (15)
RF-20	services/news.py	test_news_service.py (16)
RF-21, RF-22	agent/, services/conversation.py, preferences.py	test_agent_toolset.py (4), test_conversation_service.py (3), test_preferences_service.py (7)
RF-23, RF-24	services/share.py, export.py	test_share_service.py (7), test_export_service.py (3)
RF-26, RF-27	services/sync.py, prices.py, integrations/	test_catalog_sync.py (6), test_prices.py (5), test_tcgdex.py (9), test_pokeapi.py (7)
RF-28	seed.py	test_seed.py (17)
RNF-11	storage/	test_storage.py (12)

3

Arquitectura

3.1 Vista de contexto

El sistema es una pieza con dos públicos —una persona con un navegador y un asistente con un cliente MCP— y tres dependencias externas, dos obligatorias para tener catálogo y una opcional para tener modelo.

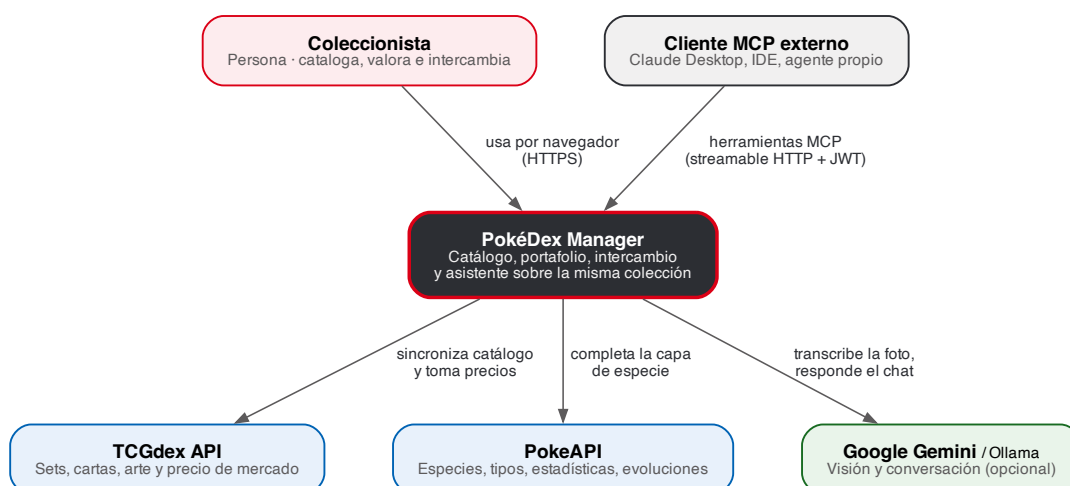


Figura 3.1 Diagrama de contexto (C4 nivel 1). Generado desde `diagramas/src/c4-contexto.dot`.

3.2 Vista de contenedores

Dos ejecutables y una base de datos. Lo que hay que mirar en la figura 3.2 es la arista roja: el navegador habla con Next.js para su sesión, pero habla con FastAPI *directamente*, cruzando origen, con un *bearer* en la cabecera. No hay reescrituras ni proxy en `next.config.ts`; esa ausencia es deliberada y es lo que obliga al esquema de token en vez de reenviar la cookie.

La autenticación cruza el límite una sola vez

Better Auth firma un JWT Ed25519 a partir de la cookie de sesión; FastAPI lo verifica *sin conexión* contra el JWKS y nunca llama de vuelta. El emisor (`AUTH_ISSUER`) es lo que el token declara y lo que el navegador ve; la URL del JWKS (`AUTH_JWKS_URL`) es cómo el contenedor de la API alcanza al de la web. En Docker son distintas a propósito: `http://localhost:3000` frente a `http://web:3000/api/auth/jwks`.

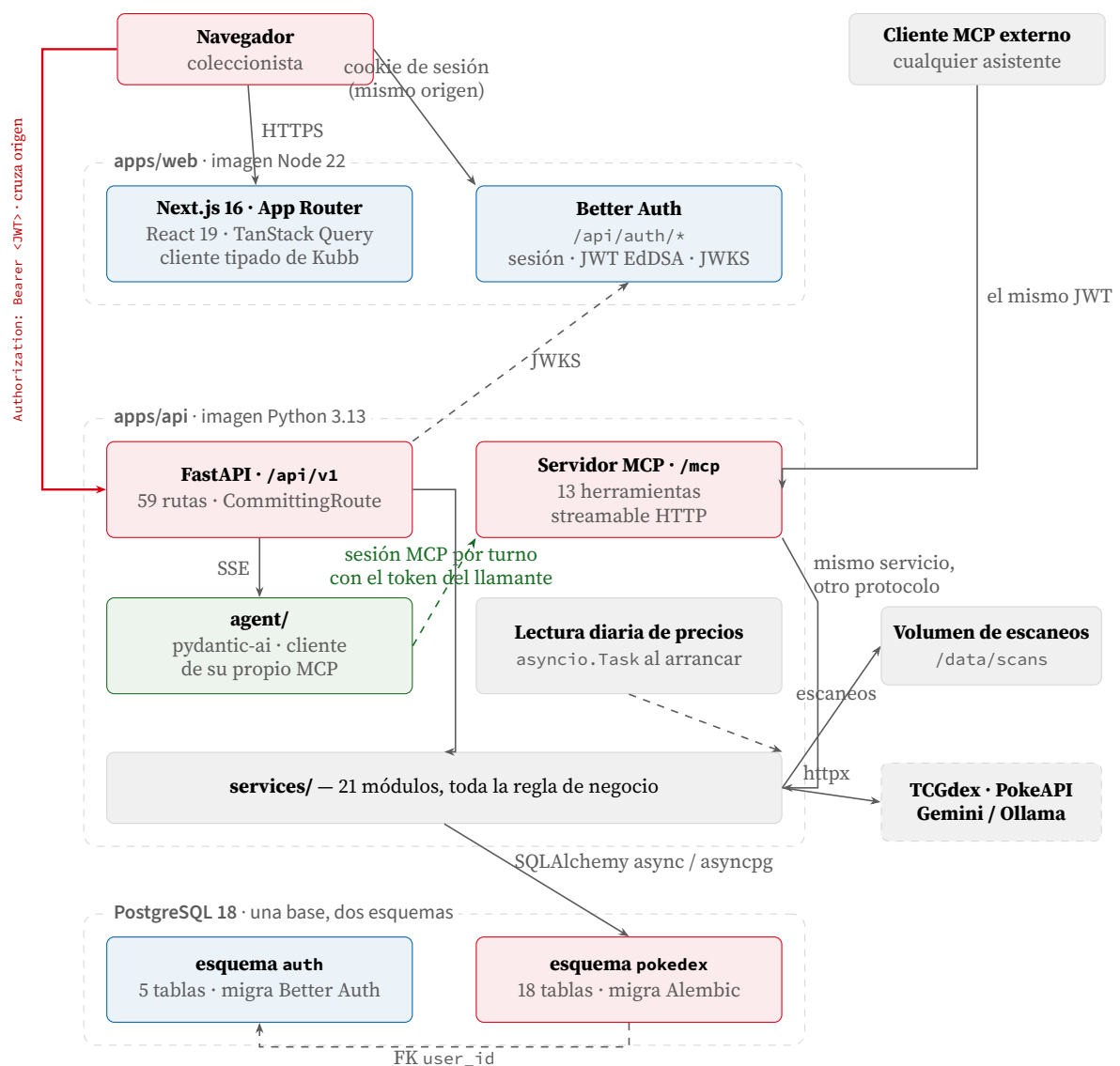


Figura 3.2 Diagrama de contenedores (C4 nivel 2). La arista roja es la importante: el navegador toma su sesión de Next.js pero llama a FastAPI directamente, cruzando origen, con un *bearer* en la cabecera.

3.3 El monorepo

Turborepo con *npm workspaces*. Cada tarea (*build*, *lint*, *typecheck*, *test*) se declara en *turbo.json* con *dependsOn*: `["^build"]` y se lanza desde la raíz; el filtrado (*-filter=web...*) es lo que permite que CI corra sólo el subárbol del frontend en un job y el de Python en otro.

```
pokedex-manager/
|- apps/
| | |- api/           FastAPI + SQLAlchemy + MCP + agente   (gestionado por uv)
| | | | |- src/pokedex/{api,services,db,agent,mcp,schemas,...}
| | | | |- alembic/    migraciones del esquema 'pokedex'
| | | | '- tests/     292 casos contra Postgres real
| | '- web/          Next.js 16 App Router                  (workspace npm)
| |   |- app/        rutas; grupo (app) = superficie autenticada
| |   |- lib/api/     cliente tipado GENERADO por Kubb (no editar a mano)
| |   |- scripts/    migrate-auth.mjs, paquete npm aparte
| |   '- test/       setup de Vitest, servidor MSW
|- packages/
| | |- ui/           25 componentes shadcn sobre Base UI + tokens de diseno
| | |- eslint-config/
| | '- typescript-config/
|- db/init/         01-schemas.sql (esquemas + extension pg_trgm)
|- .github/workflows/ci.yml
'- docker-compose.yml
```

Por qué packages/ui no se compila

El paquete se consume como fuente: no tiene *build*, exporta un archivo por subruta (*./components/**) y Next lo procesa vía *transpilePackages*. *apps/web/tsconfig.json* apunta el alias *@workspace/ui/** directamente a *packages/ui/src/**, así que no hay artefacto intermedio que pueda quedar desincronizado.

3.4 Tecnologías

Pieza	Versión	Papel
Python	≥ 3.13	Sintaxis de genéricos PEP 695 (<code>class Page[T]</code>)
FastAPI	≥ 0.141.1	Rutas, validación, OpenAPI, <code>StreamingResponse</code>
SQLAlchemy	≥ 2.0.52	ORM asíncrono, <code>Mapped[...]</code> tipado
asyncpg	≥ 0.31.0	Driver
Alembic	≥ 1.19.1	Migraciones del esquema <code>pokedex</code>
mcp	≥ 2.0.0	Servidor MCP y sesión cliente
pydantic-ai-slim	≥ 1.47.0	Agente, visión, salida estructurada
PyJWT[crypto]	≥ 2.13.0	Verificación EdDSA contra JWKS
Pillow + pillow-heif	≥ 12.3 / 1.5	Normalización de la foto (HEIC incluido)
uv	—	Resolución, bloqueo y ejecución del entorno Python
PostgreSQL	18-alpine	Base única, dos esquemas, <code>pg_trgm</code>
Node	≥ 20 (CI: 24)	Runtime del frontend
Next.js	16.2.6	App Router, <code>output: "standalone"</code>
React	19.2.4	—
TanStack Query	5.101.4	Caché de servidor en el cliente
Better Auth	1.6.27	Sesión, esquema <code>auth</code> , plugin <code>jwt()</code>
Kubb	4.39.3	Generación de tipos, clientes y hooks desde OpenAPI
Tailwind CSS	4	Sin archivo de configuración: tokens en CSS
Base UI	1.7.0	Primitivas accesibles (no Radix)
Vitest + RTL + MSW	3.2.7 / 16.3 / 2.15	Pruebas del frontend
Turborepo	2.9.18	Orquestación y caché de tareas

3.5 Reglas de arquitectura

- Una sola capa de lógica.** REST y MCP son dos superficies sobre los mismos 21 módulos de `services/`. Ninguna herramienta MCP escribe SQL propio. Un cambio de regla de negocio se hace una vez.
- La carta y la especie son capas distintas.** Una carta es una impresión; una especie es el Pokémon retratado. Se unen por el `dexId` del origen, nunca comparando nombres.
- Cada esquema tiene un dueño y una herramienta.** Better Auth migra `auth`; Alembic migra `pokedex`; ninguno toca el del otro (sección 4.1).
- La colección sólo la escribe su dueño.** Ni el agente ni aceptar un intercambio mueven cartas: lo que alguien tiene en la mano sólo lo sabe quien la tiene.
- Lo que se puede verificar se verifica en la base.** Unicidad, rangos y coherencia son restricciones, no comprobaciones en Python.

4

Modelo de datos

Una base PostgreSQL 18, dos esquemas, dos dueños. `auth` tiene cinco tablas (`user`, `session`, `account`, `verification`, `jwtks`) y las escribe Better Auth. `pokedex` tiene dieciocho tablas de dominio más `alembic_version`, y las escribe Alembic. Los modelos viven en `apps/api/src/pokedex/db/models/`.

4.1 Dos esquemas, dos herramientas de migración

La separación no es estética: cada esquema lo define el sistema que sabe cómo tiene que ser. Better Auth genera sus tablas desde su propia configuración —si mañana se añade un plugin, la tabla que hace falta la crea él— y Alembic genera las del dominio desde `Base.metadata`. El precio de mezclarlos sería que `alembic revision -autogenerate` viera tablas ausentes de su metadata y propusiera borrarlas.

Ese riesgo se cierra con dos filtros en `apps/api/alembic/env.py`, que cubren direcciones distintas:

```
DOMAIN_SCHEMA = "pokedex"

def include_name(name, type_, parent_names):
    # Without this filter autogenerate reflects Better Auth's 'auth' tables and
    # proposes dropping them, since they are absent from Base.metadata.
    if type_ == "schema":
        return name == DOMAIN_SCHEMA
    return True

def include_object(obj, name, type_, reflected, compare_to):
    # 'auth.user' is registered in Base.metadata only so cross-schema foreign
    # keys resolve; alembic must never try to create or drop it.
    if type_ == "table":
        return bool(getattr(obj, "schema", None) == DOMAIN_SCHEMA)
    return True
```

`include_name` bloquea lo que se *refleja* desde la base; `include_object` bloquea lo que viene de la metadata. La tabla `auth.user` está declarada en `db/models/external.py` como un `Table` suelto con dos columnas, únicamente para que resuelvan las claves foráneas entre esquemas.

Dos cosas no pueden crearse dentro de la cadena de migraciones, porque la primera revisión ya las necesita, y por eso se ejecutan antes:

```
await connection.execute(text(f'CREATE SCHEMA IF NOT EXISTS "{DOMAIN_SCHEMA}"'))
await connection.execute(text("CREATE EXTENSION IF NOT EXISTS pg_trgm"))
```

El `search_path` está fijado a `public`

El motor se crea con `server_settings={"search_path": "public"}`. Con el valor por defecto (`"$user", public`) el esquema `pokedex` pasaría a ser implícito y los filtros de Alembic dejarían de

discriminar. Por la misma razón `pg_trgm` se instala en `public`: un `gin_trgm_ops` instalado en otro esquema sería invisible al crear el índice.

4.2 Catálogo y colección

4.2.1 Por qué `card_price` es una fila por carta y por día

`card_price_usd` guarda *siempre* la última lectura, y eso hace el movimiento inmedible: un portafolio sin serie es un número suelto. La tabla `card_price` es la historia detrás de ese número, con clave primaria compuesta (`card_id`, `recorded_on`). La escritura es un `ON CONFLICT DO UPDATE` sobre esa misma clave, así que volver a sincronizar el mismo día *sobrescribe* el día en vez de añadir un punto: la serie se mantiene en un punto por día y el trabajo es idempotente.

4.2.2 Por qué un sobrante es `SUM(quantity) > 1`

No existe una columna «para cambiar». Un sobrante se calcula:

```
SELECT user_id, card_id, SUM(quantity) - 1 AS spare
FROM pokdex.collection_item
GROUP BY user_id, card_id
HAVING SUM(quantity) > 1
```

La primera copia es la colección; sólo lo que pasa de ahí es inventario. La consecuencia práctica es que nadie tiene que mantener un estado «disponible/reservado» coherente con las cantidades reales: al publicar, al ofertar y al aceptar se recalcula, así que el tablón nunca puede prometer una carta que ya no existe. El agrupamiento es por (`user_id`, `card_id`) y no por condición: dos copias en estados distintos siguen siendo dos copias.

4.2.3 Por qué el único de `collection_item` usa `NULLS NOT DISTINCT`

Una fila no es una carta física: es un grupo homogéneo de copias. El único cubre (`user_id`, `card_id`, `condition`, `language`, `is_graded`, `grade`), y `grade` es `NULL` para todo lo que no está gradeado. Postgres trata los `NULL` como distintos por omisión, lo que permitiría insertar el mismo grupo una y otra vez e impediría que `ON CONFLICT` disparase nunca. `postgresql_nulls_not_distinct=True` es lo que convierte el alta en una fusión real.

El coste se guarda por grupo, no por copia (`unit_cost_usd`), de modo que copias compradas a precios distintos se promedian —`_blended_cost` pondera por cantidad— en lugar de quedar como lotes separados. Si un lado no tiene coste conocido, el coste conocido carga el grupo entero.

4.3 Intercambio, conversación y mensajería

4.3.1 Una oferta tiene destinatario; una publicación, no

Esa es toda la diferencia entre `trade_offer` y `trade_listing`. Tomar una publicación escribe una `trade_offer` ya *aceptada*, con el publicador como emisor y quien la tomó como destinatario, y enlaza ambas por `trade_listing.offer_id`. Desde ese momento, un trato que empezó en el tablón y uno que empezó como propuesta son lo mismo, y quien la tomó vive en un solo sitio, así que los dos registros no pueden contradecirse.

4.3.2 Por qué la condición se copia y la cantidad no se guarda

`trade_offer_card` lleva `condition` como columna propia en vez de apuntar a la fila de colección de la que salió: una oferta es una promesa sobre una carta *en un estado*, y tiene que sobrevivir a que su dueño edite o borre esa fila. Y la condición no es un detalle —es la mayor parte del precio: una carta *near mint* y una *damaged* son la misma fila sin ella, y valen varias

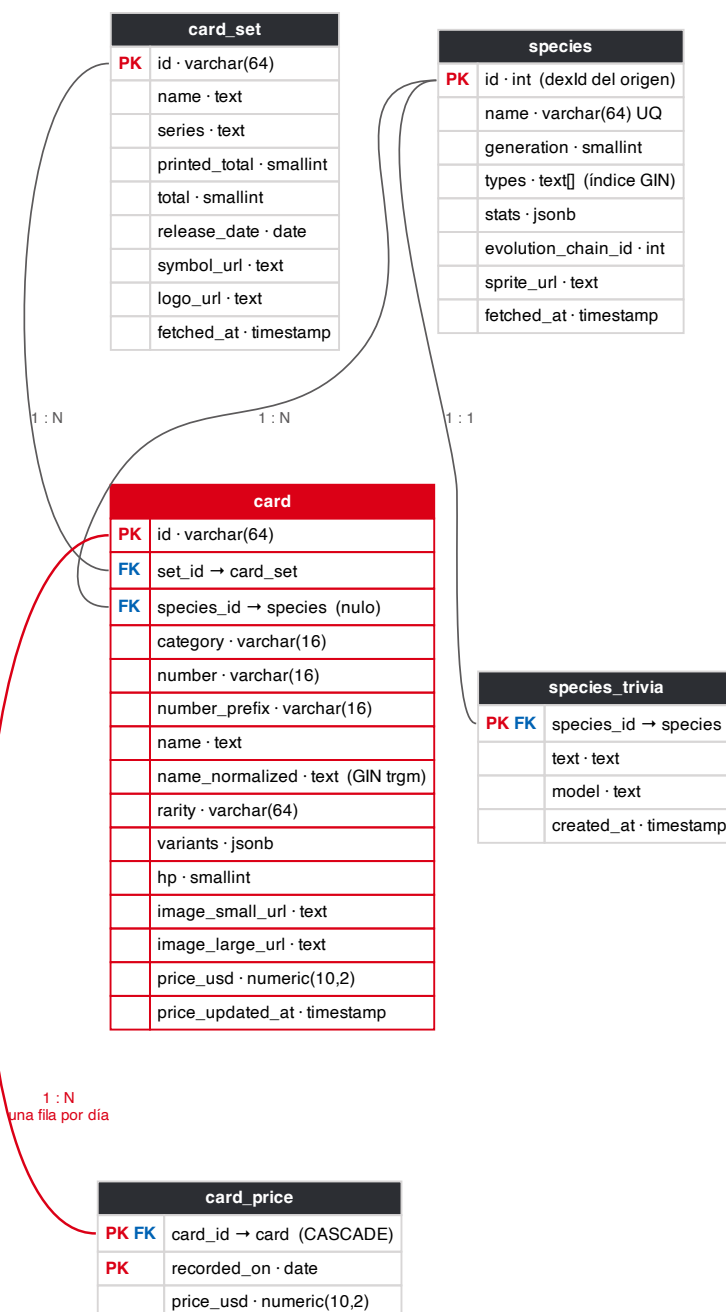


Figura 4.1 Capa de catálogo: una carta es una impresión, una especie es lo retratado, y **card_price** es la serie detrás del último precio. Generado desde `diagramas/src/er-catalogo.dot`.

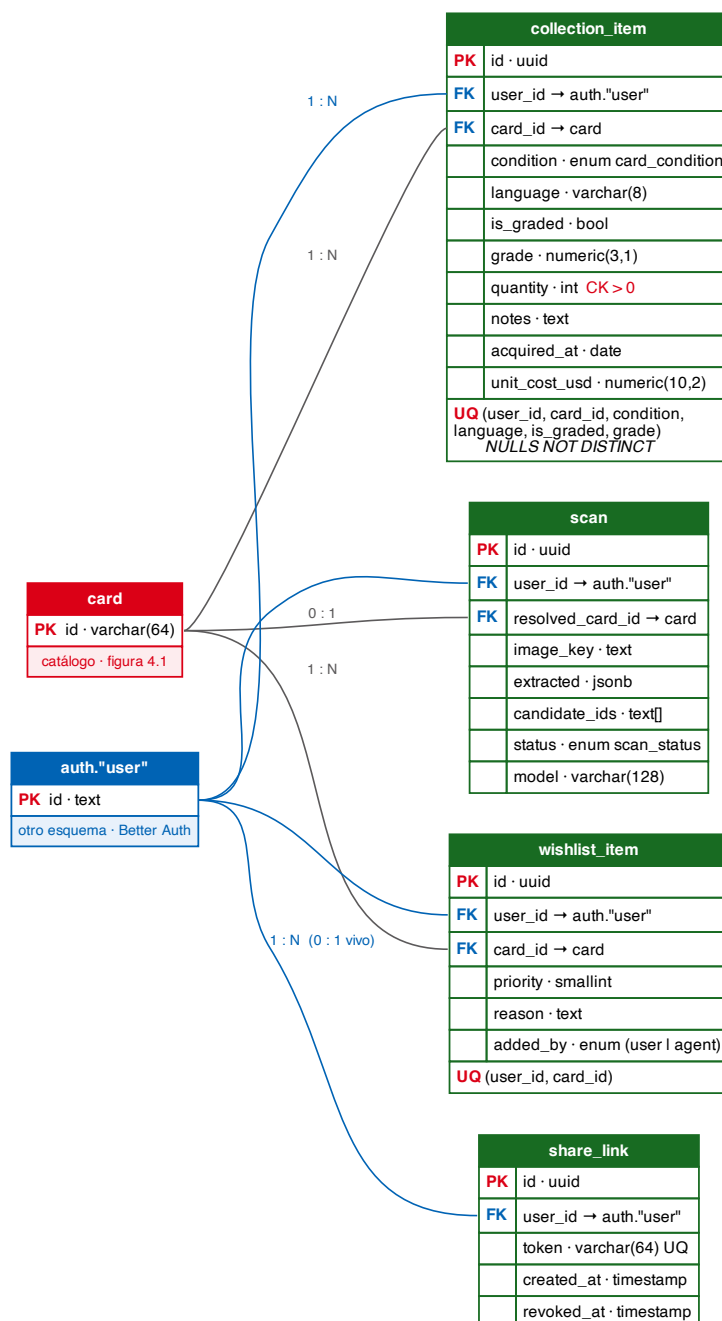


Figura 4.2 Capa de colección: todo cuelga de **auth."user"** y del catálogo, y nada de aquí es visible para otro usuario. Generado desde `diagramas/src/er-coleccion.dot`.

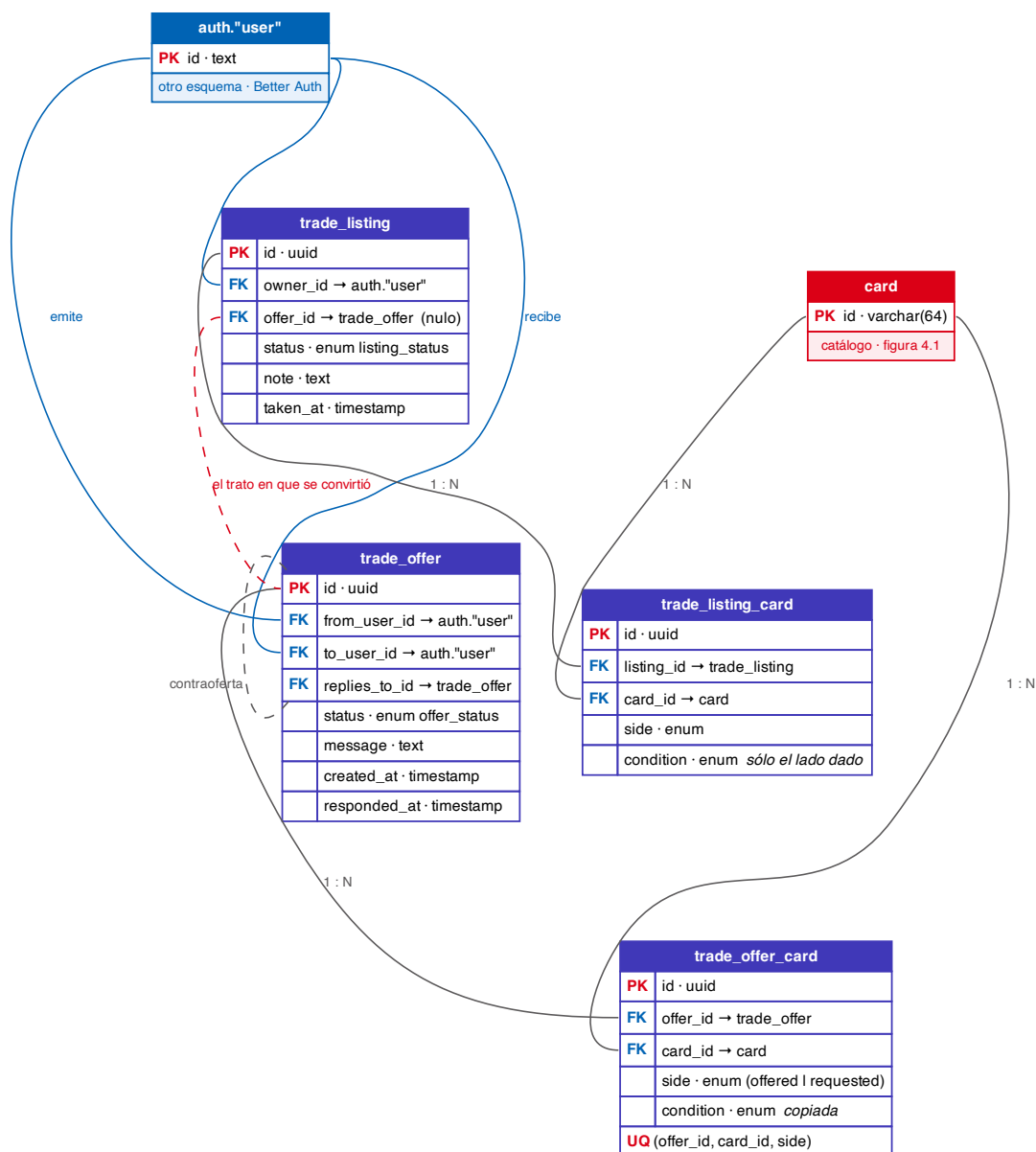


Figura 4.3 Intercambio. Una oferta tiene destinatario y una publicación no; tomarla escribe la oferta ya aceptada que las une. Generado desde `diagramas/src/er-intercambio.dot`.

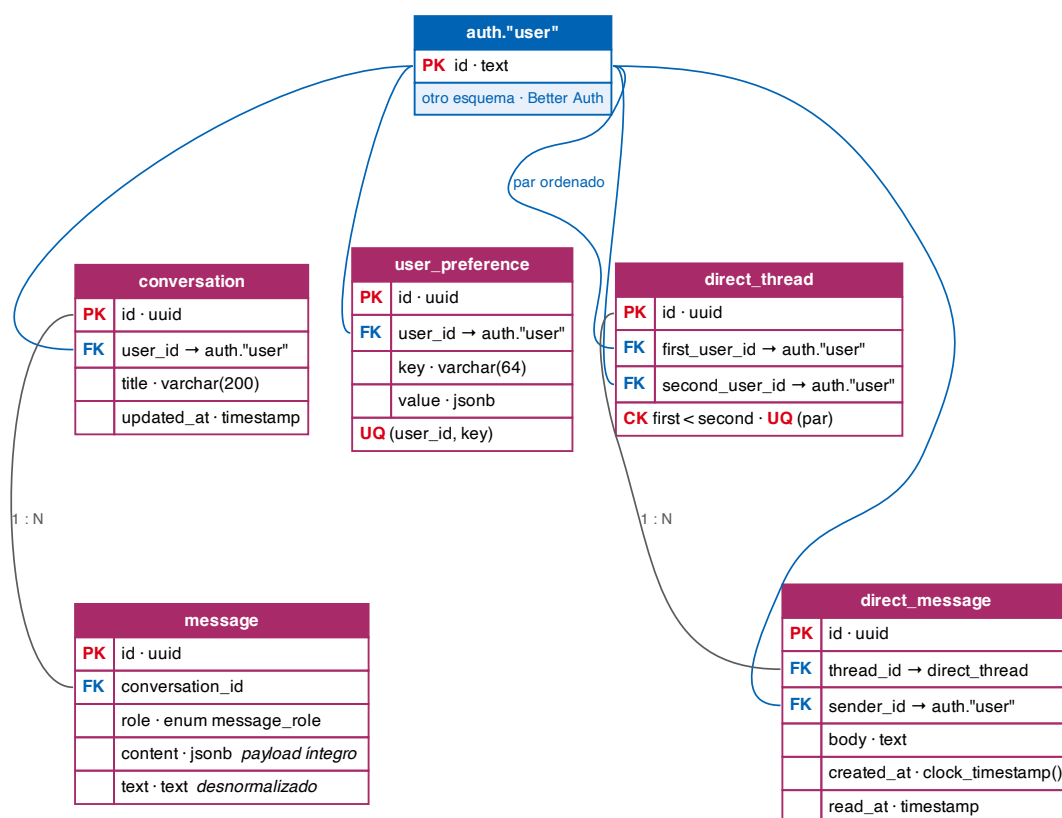


Figura 4.4 Conversación con el asistente y mensajería entre coleccionistas. Generado desde `diagramas/src/er-mensajeria.dot`.

veces la una a la otra.

La cantidad, en cambio, está deliberadamente ausente: una oferta nombra cartas, y cuántas copias cambian de manos se acuerda cuando las dos personas quedan. Anotar un número aquí parecería un compromiso que la aplicación no puede exigir a nadie.

En `trade_listing_card` sólo el lado *ofrecido* lleva condición: en qué estado llegará una carta pedida lo responde quien tome la publicación, y todavía no hay a quién preguntar.

4.3.3 Los multiplicadores de condición

Viven en `services/trade.py` y su orden de declaración es además el orden de mejor a peor que usa el resto del módulo:

Condición	Factor	Uso
<code>mint</code>	1.05	<code>adjusted(price, condition)</code> multiplica y cuantiza a dos decimales. Se aplica al valorar una oferta, una publicación y el simulador. La lista también define el orden «peor copia», que es el que se toma cuando una oferta no declara condición.
<code>near_mint</code>	1.00	
<code>lightly_played</code>	0.85	
<code>moderately_played</code>	0.70	
<code>heavily_played</code>	0.50	
<code>damaged</code>	0.35	

Son ratios del gremio, no una medición

El comentario que acompaña a la tabla lo dice sin rodeos: «estas son las proporciones con las que negocia la afición, no algo medido aquí, así que toda cifra derivada de ellas se rotula como estimación allí donde se muestra». La interfaz respeta ese contrato.

4.3.4 Un hilo es un par ordenado

`direct_thread` guarda `first_user_id` y `second_user_id` ordenados, con `CHECK first_user_id < second_user_id` y único sobre el par. Con un par emisor/receptor las mismas dos personas podrían abrir dos hilos escribiéndose a la vez, y cada lectura tendría que unir las dos mitades. No hay tabla de participantes porque no hay un tercer participante.

`direct_message.read_at` no necesita dueño: en un hilo de dos, quien no escribió el mensaje es la única persona que puede leerlo. Y su `created_at` usa `clock_timestamp()` y no `now()`, porque `now()` es el instante en que empezó la transacción: dos mensajes escritos en la misma transacción compartirían marca y el hilo se quedaría sin orden en el que leerse.

4.3.5 El historial del asistente se guarda íntegro

`message.content` es `jsonb` con el *payload* del framework tal cual, para que una conversación reanudada se reproduzca con sus llamadas a herramientas intactas en lugar de como una transcripción con pérdida. `message.text` está desnormalizado para poder listar una conversación sin deserializar cada *payload*.

`user_preference` es la memoria del agente: una fila por clave, única por usuario. Las claves con prefijo `sys.` son internas y no se exponen por la API; `sys.notifications_seen_at` es la que sostiene el estado «visto» del feed de novedades.

4.4 Índices y restricciones que importan

Objeto	Tipo	Por qué existe
<code>ix_card_name_normalized_trgm</code>	GIN <code>gin_trgm_ops</code>	La resolución de escaneos compara nombres con el operador <code>%</code> ; sin este índice sería un recorrido completo del catálogo.
<code>ix_species_types</code>	GIN sobre <code>text[]</code>	Filtrar la colección por tipo.
<code>ix_message_conversation_created</code>	B-tree compuesto	Una conversación se lee siempre en orden y nunca como feed global.
<code>ix_direct_message_thread_created</code>	B-tree compuesto	Lo mismo para un hilo directo.
<code>uq_collection_item_group</code>	UNIQUE <i>nulls not distinct</i>	Convierte el alta en fusión.
<code>ck_collection_item_quantity_positive</code>	CHECK	Un grupo con cero copias no es un grupo.
<code>ck_collection_item_grade_only_when_graded</code>	CHECK	<code>grade</code> y <code>is_graded</code> no pueden discrepar.
<code>ck_direct_thread_ordered</code>	CHECK	Hace verificable el par ordenado.
<code>uq_trade_offer_card</code>	UNIQUE	La misma carta no puede repetirse en el mismo lado de una oferta.

4.5 Migraciones

Catorce revisiones en `apps/api/alembic/versions/`, encadenadas desde `bb4e34c86509_init`. La tabla de versión vive en el propio esquema (`version_table_schema="pokedex"`) y `compare_type` está activo, así que un cambio de tipo se detecta en el autogenerado.

```
uv run alembic upgrade head          # aplicar
uv run alembic revision --autogenerate -m "mensaje en ingles"
uv run alembic downgrade -1         # revertir una
```

Orden de arranque

Las tablas de dominio tienen claves foráneas hacia `auth."user"`, que crea Better Auth. En un despliegue nuevo los dos servicios arrancan a la vez, así que la imagen de la API reintenta `alembic upgrade head` hasta treinta veces con cinco segundos de espera en lugar de entrar en bucle de caída.

5

Backend

FastAPI sobre SQLAlchemy asíncrono, con dos superficies —REST y MCP— apoyadas en una sola capa de servicios. 87 archivos bajo `mypy -strict`.

5.1 Capas

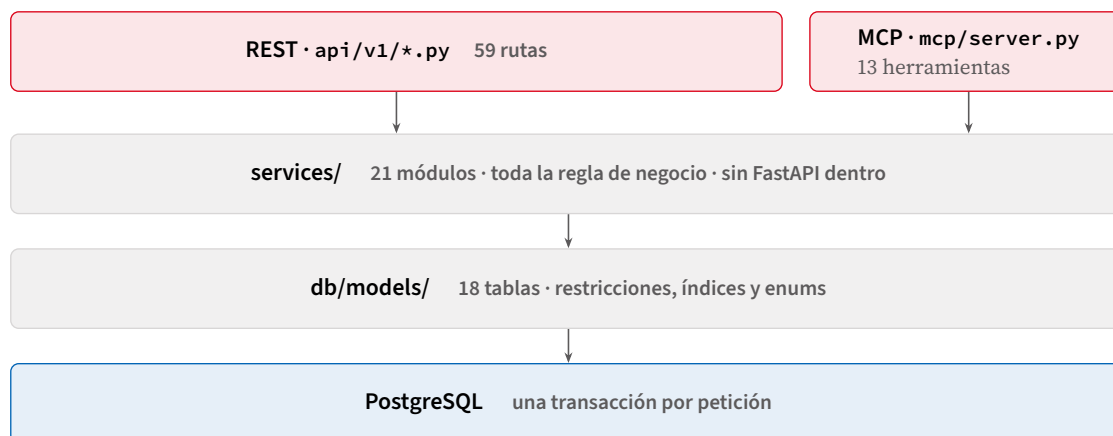


Figura 5.1 Las capas del backend. Una regla se escribe una vez y las dos superficies la comparten; ningún módulo de `services/` importa FastAPI.

5.2 `CommittingRoute`: confirmar antes de responder

Es la pieza menos evidente del backend y merece el detalle. FastAPI ejecuta la mitad de una dependencia `yield` que va *después* del `yield` cuando la respuesta ya salió. Con una sesión por petición, eso significa que el `commit` aterriza cuando el cliente ya es libre de actuar sobre el resultado. Un cliente que refresca al éxito compite entonces consigo mismo: la lectura puede llegar a la base antes que el `commit` y volver sin el cambio, lo que se lee como una mutación que no hizo nada hasta recargar la página.

```
class CommittingRoute(APIRoute):
    """Commits the request's transaction before its response leaves."""

    def get_route_handler(self):
        handler = super().get_route_handler()

        async def commit_before_responding(request: Request) -> Response:
            response = await handler(request)
            session = getattr(request.state, "db", None)
            if session is not None:
                await session.commit()
            return response
```

```
return commit_before_responding
```

Funciona porque `get_db` publica la sesión en la petición (`request.state.db = session`) antes de cederla. Los trece routers de `api/v1/` declaran `route_class=CommittingRoute`; el commit dentro de `get_db` se queda como red de seguridad para lo que nunca llegue aquí.

Consecuencia sobre el ORM

El commit ocurre después de que el manejador retorna pero *antes* de serializar la respuesta, así que cualquier atributo cargado perezosamente ya tiene que estar cargado. Por eso `SessionFactory` se construye con `expire_on_commit=False` y las relaciones sensibles se declaran `lazy="raise"`: un acceso perezoso accidental falla ruidosamente en vez de emitir una consulta silenciosa.

5.3 Paginación tipada: `Page[T]`

```
class Page[T](BaseModel):
    items: List[T]
    total: int
    limit: int
    offset: int
```

Cuatro líneas, sintaxis de genéricos de PEP 695. Lo relevante es que *atraviesa el generador*: FastAPI escribe en el OpenAPI un esquema concreto por instancia (`Page[CollectionItemView]`, `Page[TradeMatch]`...), Kubb lo convierte en un tipo TypeScript por endpoint, y el frontend recibe `items`, `total`, `limit` y `offset` tipados sin que nadie los escriba dos veces.

Los filtros y la paginación se inyectan como modelos Pydantic (`Annotated[CollectionFilters, Depends()]`), con los toques en el propio modelo: `limit=50`, `ge=1`, `le=200` en colección y mercado, `limit=20`, `le=100` en posiciones.

5.4 Autenticación

```
# Pinned rather than read from the token header, which would allow algorithm
# confusion. Better Auth's jwt plugin issues Ed25519 keys.
ALGORITHMS = ["EdDSA"]

async def verify_token(token: str) -> AuthenticatedUser:
    ...
    # PyJWKClient fetches over blocking urllib, so keep it off the event loop.
    signing_key = await to_thread.run_sync(client.get_signing_key_from_jwt, token)
    payload = jwt.decode(token, signing_key.key, algorithms=ALGORITHMS,
                        issuer=settings.auth_issuer, audience=settings.auth_audience)
    subject = payload.get("sub")
    if not subject:
        raise jwt.InvalidTokenError("token has no subject claim")
    return AuthenticatedUser(id=subject, email=payload.get("email"))
```

Tres detalles que un revisor busca: el algoritmo está *fijado* y no se lee de la cabecera del token —leerlo abre la puerta a la confusión de algoritmos—; la descarga del JWKS es bloqueante en PyJWT y por eso se saca del bucle de eventos con `anyio.to_thread`; y la caché de claves es la del propio `PyJWKClient`, mantenida viva por un `@lru_cache` sobre la fábrica, de modo que hay un cliente por proceso.

`Caller` lleva el usuario y el token, porque el endpoint de chat necesita los dos: el identificador para acotar sus filas y el token para abrir una sesión MCP como ese mismo usuario en lugar de como el servicio.

5.5 Catálogo de endpoints

Todas las rutas cuelgan de `/api/v1`. El `operationId` es exactamente el nombre de la función manejadora, porque el generador de ids de FastAPI produciría, si no, ganchos como `useListCollectionApiV1CollectionGet`:

```
def operation_id(route: APIRoute) -> str:
    return route.name
```

Tabla 5.1. Catálogo completo de rutas de `/api/v1`

Método	Ruta	operationId	Respuesta
GET	<code>/me</code>	<code>me</code>	<code>MeResponse</code>
GET	<code>/preferences</code>	<code>list_preferences</code>	<code>list[PreferenceView]</code>
DELETE	<code>/preferences/{key}</code>	<code>forget_preference</code>	204
POST	<code>/advice/trade</code>	<code>propose_trade</code>	<code>TradeAdvice</code> · 503/404
GET	<code>/catalog/cards</code>	<code>search_cards</code>	<code>list[CardView]</code>
GET	<code>/catalog/cards/{id}</code>	<code>get_card</code>	<code>CardView</code>
GET	<code>/catalog/cards/{id}/context</code>	<code>card_market_context</code>	<code>CardMarketContext</code>
GET	<code>/catalog/species</code>	<code>search_species</code>	<code>list[SpeciesView]</code>
GET	<code>/catalog/species/{id}/trivia</code>	<code>species_trivia</code>	<code>TriviaView</code> None
GET	<code>/catalog/species/{id}/evolutionsspecies_evolutions</code>		<code>list[EvolutionMemberView]</code>
GET	<code>/catalog/owned-ids</code>	<code>owned_card_ids</code>	<code>list[str]</code>
GET	<code>/catalog/market</code>	<code>market_cards</code>	<code>Page[MarketCardView]</code>
GET	<code>/catalog/market/summary</code>	<code>market_summary</code>	<code>MarketSummary</code>
GET	<code>/catalog/market/positions</code>	<code>market_positions</code>	<code>Page[PositionView]</code>
GET	<code>/catalog/market/concentration</code>	<code>market_concentration</code>	<code>PortfolioConcentration</code>
GET	<code>/catalog/market/sets</code>	<code>market_sets</code>	<code>list[SetMarketView]</code>
POST	<code>/catalog/market/simulate</code>	<code>simulate_trade</code>	<code>TradeSimulation</code> · 404/422
GET	<code>/chat</code>	<code>list_conversations</code>	<code>list[ConversationView]</code>
GET	<code>/chat/{id}</code>	<code>get_conversation</code>	<code>ConversationDetail</code>
POST	<code>/chat</code>	<code>send_message</code>	SSE · 503 sin modelo
GET	<code>/collection</code>	<code>list_collection</code>	<code>Page[CollectionViewItemView]</code>
POST	<code>/collection</code>	<code>add_card</code>	201 <code>CollectionViewItemView</code>
GET	<code>/collection/activity</code>	<code>collection_activity</code>	<code>list[ActivityEntry]</code>
GET	<code>/collection/export</code>	<code>export_collection</code>	CSV o JSON adjunto
GET	<code>/collection/{id}</code>	<code>get_item</code>	<code>CollectionViewItemView</code>
PATCH	<code>/collection/{id}</code>	<code>update_item</code>	<code>CollectionViewItemView</code>
DELETE	<code>/collection/{id}</code>	<code>remove_item</code>	204

Continúa en la página siguiente

Tabla 5.1. Catálogo completo de rutas de `/api/v1` (continuación)

Método	Ruta	operationId	Respuesta
GET	<code>/messages</code>	<code>list_threads</code>	<code>Page[ThreadView]</code>
GET	<code>/messages/with/{id}</code>	<code>get_thread_with</code>	<code>ThreadView</code>
GET	<code>/messages/{id}/messages</code>	<code>list_thread_messages</code>	<code>Page[DirectMessageView]</code>
POST	<code>/messages</code>	<code>send_direct_message</code>	201 <code>DirectMessageView</code>
POST	<code>/messages/{id}/read</code>	<code>mark_thread_read</code>	<code>ThreadView</code>
GET	<code>/news</code>	<code>news_feed</code>	<code>NewsFeed</code>
POST	<code>/news/seen</code>	<code>mark_news_seen</code>	204
POST	<code>/scans</code>	<code>create_scan</code>	201 <code>ScanResult</code> (multipart)
GET	<code>/scans/{id}/image</code>	<code>get_scan_image</code>	<code>image/jpeg</code>
POST	<code>/scans/{id}/confirm</code>	<code>confirm_scan</code>	201 <code>CollectionView</code>
GET	<code>/stats</code>	<code>collection_stats</code>	<code>CollectionStats</code>
GET	<code>/gaps</code>	<code>list_gaps</code>	<code>list[SetGap]</code>
GET	<code>/wishlist</code>	<code>list_wishlist</code>	<code>list[WishlistItemView]</code>
POST	<code>/wishlist</code>	<code>add_to_wishlist</code>	201 <code>WishlistItemView</code>
DELETE	<code>/wishlist/{id}</code>	<code>remove_from_wishlist</code>	204
GET	<code>/share</code>	<code>get_share_link</code>	<code>ShareLinkView</code> None
POST	<code>/share</code>	<code>create_share_link</code>	201 <code>ShareLinkView</code>
DELETE	<code>/share</code>	<code>revoke_share_link</code>	204
GET	<code>/public/{token}</code>	<code>read_shared_collection</code>	<code>PublicCollection</code> · sin auth
GET	<code>/trades</code>	<code>list_trades</code>	<code>Page[TradeMatch]</code>
GET	<code>/trades/collectors</code>	<code>list_collectors</code>	<code>Page[CollectorView]</code>
GET	<code>/trades/collectors/{id}</code>	<code>get_collector</code>	<code>CollectorProfile</code>
GET	<code>/trades/collectors/{id}/spares</code>	<code>list_spares</code>	<code>Page[SpareCard]</code>
GET	<code>/trades/offers</code>	<code>list_offers</code>	<code>Page[TradeOfferView]</code>
POST	<code>/trades/offers</code>	<code>create_offer</code>	201 · 422
POST	<code>/trades/offers/{id}/counter</code>	<code>counter_offer</code>	201 · 404/409
POST	<code>/trades/offers/{id}/respond</code>	<code>respond_to_offer</code>	200 · 404/409
POST	<code>/trades/offers/{id}/withdraw</code>	<code>withdraw_offer</code>	200 · 404/409
GET	<code>/trades/listings</code>	<code>list_listings</code>	<code>Page[TradeListingView]</code>
POST	<code>/trades/listings</code>	<code>publish_listing</code>	201 · 422
POST	<code>/trades/listings/{id}/accept</code>	<code>accept_listing</code>	<code>TradeOfferView</code> · 404/409
POST	<code>/trades/listings/{id}/cancel</code>	<code>cancel_listing</code>	<code>TradeListingView</code> · 404/409

Fuera de `/api/v1` sólo hay `GET /health`, la referencia interactiva en `/scalar` (excluida del esquema) y la aplicación MCP montada en la raíz. `GET /api/v1/public/{token}` es la única ruta

v1 sin autenticación, y responde 404 a un enlace revocado exactamente igual que a uno que nunca existió, para que no se puedan distinguir.

5.6 La resolución de un escaneo

El escaneo separa tres decisiones: el modelo lee lo impreso, el código decide qué carta es, y la persona decide qué se guarda. La segunda es la interesante, y es determinista: una consulta SQL que *puntuá* en vez de filtrar.



Figura 5.2 Los pesos de `services/resolve.py` y los umbrales que convierten una puntuación en un estado.

```
# Every signal contributes; none vetoes. A hard filter on a misread field would
# discard the right card.
W_SET_TOTAL = 0.35
W_NUMBER    = 0.35
W_NAME      = 0.25
W_HP        = 0.05

PLAUSIBLE = 0.4 # above this a candidate is worth showing at all
CONFIDENT = 0.7 # above this, and clear of the runner-up, it is settled
# A margin, not a ceiling on the runner-up: in a full catalog near-miss scores
# around 0.45 are always present and would make every scan ambiguous.
MARGIN = 0.3
MAX_CANDIDATES = 8
```

Dos propiedades importan. La primera: el estrechamiento de filas es una *unión* (`or` (`*conditions`)), nunca una intersección, así que un HP mal leído cuesta su peso —cinco centésimas— en lugar de descartar la carta correcta. La segunda: el estado `resolved` exige puntuación alta y distancia sobre el segundo; sin ese margen, un catálogo completo siempre tiene casi-aciertos alrededor de 0.45 y todo escaneo saldría ambiguo.

5.7 La lectura diaria de precios

No hay planificador. Al arrancar el proceso se crea una tarea suelta, condicionada por `PRICE_REFRESH_ON_START` (verdadero por omisión), y se cancela al apagar:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    async with mcp_server.session_manager.run():
        task = (asyncio.create_task(take_todays_prices())
                if get_settings().price_refresh_on_start else None)
        yield
        if task is not None:
            task.cancel()
        await engine.dispose()
```

La deduplicación es por día de calendario y ocurre dos veces. Antes de empezar, `refresh_prices` comprueba si ya hay lecturas de hoy y se retira (`skipped=True`); al escribir, `record_prices` hace `ON CONFLICT (card_id, recorded_on) DO UPDATE`. Reiniciar el servicio cinco veces en un día deja una sola lectura por carta. La tarea abre su propia sesión (no la de una petición),

confirma ella misma y nunca puede tumbar el arranque: «un catálogo inalcanzable es una serie desactualizada, no una aplicación rota».

Por qué el gestor de sesiones MCP se arranca aquí

El `lifespan` de una sub-aplicación montada nunca se ejecuta, así que el anfitrión tiene que arrancar el `session_manager` del servidor MCP. Sin esa línea, la primera petición a `/mcp` falla con un error que no apunta ni de lejos al `lifespan`.

5.8 El servidor MCP

Se monta en la raíz y en último lugar, porque lleva su propia ruta `/mcp` más los metadatos de recurso OAuth que el descubrimiento espera encontrar en la raíz del dominio; Starlette empareja en orden, así que todas las rutas anteriores siguen ganando.

Su verificador de tokens delega en el mismo `verify_token` del REST: «el servidor MCP es la misma superficie de datos vista desde otro protocolo, no una segunda puerta con sus propias reglas». Una herramienta que no puede resolver a su usuario lanza `UnauthenticatedError` en lugar de recurrir a un usuario por defecto.

Herramienta	Naturaleza	Servicio que usa
<code>search_cards</code>	lectura	<code>catalog.search_cards</code>
<code>get_card_details</code>	lectura	<code>catalog.get_card</code>
<code>get_collection</code>	lectura	<code>collection.list_items</code>
<code>collection_stats</code>	lectura	<code>stats.collection_stats</code>
<code>market_summary</code>	lectura	<code>market.summary + set_breakdown</code>
<code>cheapest_missing</code>	lectura	<code>market.cheapest_missing</code>
<code>find_trades</code>	lectura	<code>trade.match_page</code>
<code>my_trade_offers</code>	lectura	<code>trade.list_offers</code>
<code>find_gaps</code>	lectura	<code>gaps.find_gaps</code>
<code>get_wishlist</code>	lectura	<code>wishlist.list_items</code>
<code>suggest_card</code>	escritura	<code>wishlist.add(added_by=AGENT)</code>
<code>remember</code>	escritura	<code>preferences.remember</code>
<code>forget</code>	escritura	<code>preferences.forget</code>

Trece herramientas, tres escriben, ninguna toca la colección

`suggest_card` sólo puede proponer una carta para la lista de deseos. Su documentación fija el límite: «esta es la única escritura disponible aquí, y deliberadamente no puede tocar la colección: eso registra qué cartas posee físicamente la persona, y nada salvo ella manipulando una carta debería cambiarlo». `remember` y `forget` escriben la memoria del agente, no datos de dominio.

5.9 El agente

Un agente nuevo por turno, atado a la sesión MCP autenticada de ese turno:

```
TURN_LIMITS = UsageLimits(tool_calls_limit=8, request_limit=10)
```

```
def build_agent(session: ClientSession, model: str, known: str = "") -> Agent[None, str]:
```

```
"""A fresh agent per turn, bound to that turn's authenticated MCP session."""  
return Agent(build_model(model), instructions=SYSTEM_PROMPT + known,  
             toolsets=[McpToolset(session)],  
             model_settings=ModelSettings(temperature=0.2))
```

El *toolset* está escrito a mano porque el cliente MCP propio de pydantic-ai apunta al SDK 1.x y no importa contra 2.0, que renombró los campos del protocolo. Si una herramienta devuelve error, el adaptador lanza `ModelRetry` para que el modelo se corrija en lugar de romper el turno.

El streaming vive en el endpoint, no en el paquete del agente: SSE con eventos `conversation`, `delta`, `tool`, `done` y `error`. Escucha `PartStartEvent` y `PartDeltaEvent` —escuchar sólo los deltas se come las primeras palabras de cada respuesta— y el cuerpo abre su propia sesión de base de datos, porque un cuerpo en streaming corre después de que el endpoint ya retornó y una sesión de petición estaría cerrándose debajo. Si el turno falla, se revierte entero: «medio turno persistido no vale nada».

Los modelos se construyen con credenciales explícitas (`agent/models.py`), no leídas del entorno del proceso, «porque eso haría de `.env` y `os.environ` dos fuentes de verdad para la misma clave». Un nombre con prefijo `ollama:` se sirve por la API compatible con OpenAI y, en visión, se le pide JSON por *prompt* (`PromptedOutput`) porque los modelos abiertos no implementan llamada a herramientas.

5.10 Almacenamiento de imágenes

`StorageBackend` es un `Protocol` con `put`, `get` y `delete`; la única implementación es un volumen de Docker, «en vez de MinIO, que añadiría un cuarto contenedor a algo que sólo corre en local». La clave de un escaneo es `"{user_id}/{scan_id}.jpg"`: la propiedad es visible en la propia ruta y borrar un usuario se convierte en borrar un directorio. `_resolve` resuelve ambos extremos y rechaza cualquier clave que se escape de la raíz, que es la comprobación de travesía que además sobrevive a los enlaces simbólicos.

`storage/images.py` valida por *firma de bytes* y no por extensión ni `Content-Type` —los dos los aporta quien sube—, acepta HEIC porque un iPhone dispara HEIC por omisión, corrige la orientación EXIF, reescala a 1600 px de lado largo y recodifica a JPEG con calidad 88. La imagen almacenada y la que se manda al modelo son los mismos bytes, así que lo que un revisor vea después es exactamente lo que el modelo vio.

Las imágenes de escaneo no se limpian

No existe trabajo de recolección: cada escaneo deja un JPEG en el volumen y ahí se queda mientras exista la fila de `scan`. En un despliegue de larga vida hay que prever un borrado periódico, o bien un backend de almacenamiento con política de expiración; la costura para lo segundo ya está puesta en `StorageBackend`.

5.11 Configuración

Variable	Por omisión	Controla
DATABASE_URL	obligatoria	DSN asyncpg
CORS_ORIGINS	http://localhost:3000	Orígenes permitidos, separados por coma
AUTH_JWKS_URL	obligatoria	Dónde alcanza la API el JWKS
AUTH_ISSUER	obligatoria	iss esperado, y emisor MCP
AUTH_AUDIENCE	obligatoria	aud esperado
MCP_RESOURCE_URL	...:8010/mcp	URL anunciada al descubrimiento externo
MCP_INTERNAL_URL	vacía	Por dónde se conecta el agente interno
STORAGE_ROOT	./data/scans	Raíz del almacén de imágenes
PRICE_REFRESH_ON_START	true	La lectura de precios al arrancar
AGENT_MODEL	google-gla: gemini-flash-latest	Chat, consejo de intercambio y reseñas
VISION_MODEL	google-gla: gemini-flash-latest	Transcripción de la foto
GOOGLE_API_KEY	vacía	Credencial y interruptor: sin ella agent_enabled es falso
OLLAMA_URL	http://localhost:11434/	Proveedor local alternativo

6

Frontend

Next.js 16 con App Router, React 19 y TanStack Query. El cliente HTTP no se escribe: se genera del OpenAPI de la API.

6.1 Estructura de rutas

Un solo grupo de rutas, (**app**), tres *layouts* y una regla simple: toda la superficie autenticada es cliente y consulta con TanStack Query; la única página con obtención de datos en servidor es la vista pública compartida.

app/	
- layout.tsx	servidor lang="es", fuentes, providers
- page.tsx	servidor redirect("/collection")
- sign-in/page.tsx	cliente
- s/[token]/page.tsx	servidor RSC asincrono, SIN autenticacion
- api/auth/[...all]/route.ts	manejador de ruta -> Better Auth
‘- (app)/	la superficie autenticada
- layout.tsx	cliente el porton de sesion
- collection/[id add]	cliente
- cards/[cardId]	cliente
- chat, scan, stats, compare	cliente
- collectors/[id]	cliente
- trades/[new publish]	cliente
- messages/[partnerId]	cliente layout propio: la lista sobrevive
‘- notifications, profile	cliente

No hay **middleware.ts**

La redirección a **/sign-in** vive en un efecto de `app/(app)/layout.tsx`. Es decir: el HTML del grupo (**app**) se sirve a cualquiera, y lo que protege los *datos* es que la API rechace toda petición sin JWT válido, no el frontend. Es una decisión coherente con el modelo —el frontend no es una frontera de confianza— pero conviene tenerla explícita.

La página pública `app/s/[token]/page.tsx` es la excepción: hace **fetch** en servidor contra `API_INTERNAL_URL` —la dirección del contenedor, no la del navegador— con **cache: "no-store"**, y llama a `notFound()` si la respuesta no es correcta.

6.2 Autenticación en el cliente

La cookie de sesión nunca sale del origen de Next. El recorrido completo:

1. El navegador guarda la cookie de Better Auth para **localhost:3000**.
2. `getAccessToken()` llama a `/api/auth/token` con **credentials: "include"** —mismo origen, la cookie viaja— y recibe un JWT.
3. Ese JWT se cachea en un módulo y se manda como **Authorization: Bearer ...** a FastAPI, en

otro origen.

4. FastAPI lo verifica contra el JWKS. Nunca ve la cookie.

```
let cachedToken: string | null = null;

export async function getAccessToken(): Promise<string | null> {
  if (cachedToken) return cachedToken;
  // Better Auth mints the JWT from the session cookie; the API never sees the cookie.
  const response = await fetch("/api/auth/token", { credentials: "include" });
  if (!response.ok) return null;
  const { token } = (await response.json()) as { token?: string };
  cachedToken = token ?? null;
  return cachedToken;
}
```

`clearAccessToken()` se invoca al iniciar sesión, al cerrarla y al cambiar de cuenta desde el perfil.

6.3 El envoltorio del cliente generado

Kubb genera funciones que aceptan un `client`; la aplicación inyecta el suyo, que hace tres cosas y ninguna más:

```
const response = await client<TResponseData, TError, TRequestData>({
  ...config,
  headers: {
    ...(isJsonBody ? { "Content-Type": "application/json" } : {}),
    ...(config.headers as Record<string, string> | undefined),
    ...(token ? { Authorization: `Bearer ${token}` } : {}),
  },
});

if (response.status >= 400) {
  throw new ApiError(response.status, detailOf(response.data));
}
```

El orden de los tres *spreads* es intencionado y está cubierto por pruebas: `Content-Type` va primero para que quien llama pueda sobrescribirlo, `Authorization` va último para que un encabezado obsoleto de quien llama no pueda desplazar al token vivo, y un cuerpo `FormData` se queda sin `Content-Type` a propósito, para que el navegador escriba el *boundary* del multipart.

`ApiError` traduce las dos formas que FastAPI usa para `detail` —cadena, o lista cuyo primer elemento trae `msg`— y cae a un mensaje en español cuando el cuerpo no es ninguna de las dos.

El envoltorio se inyecta por llamada

`kubb.config.ts` sólo elige `fetch` sobre `axios`; el envoltorio se pasa en cada sitio de llamada como `{ client: { client: apiClient } }`. Aparece en 39 archivos. Olvidarlo no rompe la compilación: simplemente salta el token y el manejo de errores.

6.4 Regeneración del cliente tipado

```
# 1. la API tiene que estar sirviendo, porque la entrada por omision es su URL
npm run dev                # o docker compose up -d api

# 2. desde apps/web
npx kubb generate           # o: OPENAPI_URL=./openapi.json npx kubb generate
```

`kubb.config.ts` declara cinco plugins y cuatro destinos bajo `apps/web/lib/api/: types/` (tipos TypeScript), `zod/` (esquemas de validación), `clients/` (una función `fetch` por operación) y `hooks/` (un `useX` y un `useXSuspense` por operación). `output.clean: true` borra el directorio antes de

escribir, por lo que cualquier retoque manual se pierde; cada archivo generado lleva escrito «Do not edit manually».

Detalles que ahorran una tarde

No hay script de `npm` para esto: la invocación es el binario de `@kubb/cli`, que sí está en `devDependencies`. El árbol generado *se versiona* en git, y está excluido de las pruebas por `vitest.config.ts`. La opción `extension: { ".ts": "" }` existe porque Kubb emite la extensión en las importaciones y TypeScript la rechaza sin `allowImportingTsExtensions`.

6.5 TanStack Query

```
export function QueryProvider({ children }: { children: ReactNode }) {
  const [queryClient] = useState(() => new QueryClient());
  return <QueryClientProvider client={queryClient}>{children}</QueryClientProvider>;
}
```

Sin opciones. El comportamiento de producción es por tanto el de fábrica de TanStack Query v5: `staleTime: 0`, `gcTime` de cinco minutos, tres reintentos con retroceso exponencial y `refetch` al enfocar la ventana, al montar y al reconectar. El inicializador perezoso del `useState` es la protección estándar del App Router contra compartir un cliente entre peticiones.

Que `staleTime` sea cero es precisamente lo que hace importante a `CommittingRoute` (sección 5): una mutación seguida de invalidación provoca un `refetch` inmediato, y ese `refetch` no puede adelantarse al `commit`.

6.6 La URL como estado

`apps/web/lib/url-state.ts` evita deliberadamente `useRouter` y `useSearchParams`:

`router.replace` es una navegación aunque sólo se mueva un parámetro de consulta: Next vuelve a pedir el payload de la ruta y el árbol se vuelve a montar, así que cada clic en un filtro parpadeaba con un esqueleto y volvía a pedir lo que ya estaba en pantalla. La History API escribe la misma URL sin ida y vuelta... El precio es que el estado ahora es de React, así que todo lo que lea la URL tiene que pasar por este hook.

Parte de la API	Comportamiento
<code>useUrlState(initial)</code>	Devuelve <code>[params, set].params</code> es un <code>URLSearchParams</code> sembrado desde <code>window.location.search</code> (a prueba de SSR) y completado con <code>initial</code> sólo en las claves que la URL no traiga.
<code>set(next)</code>	Fusiona; <code>undefined</code> o <code>""</code> borran la clave; escribe con <code>history.replaceState</code> , sin entrada de historial y sin navegación.
<code>popstate</code>	Un escucha relee la URL, así que el botón «atrás» funciona; se retira al desmontar.

Lo usan diez pantallas. El patrón habitual es fijar un filtro y limpiar la página en la misma escritura: `setParam({ q: next, p: undefined })`.

6.7 El sistema de diseño

`packages/ui` son 25 componentes shadcn construidos sobre **Base UI**, no sobre Radix, más un archivo de `tokens`. Tailwind 4 sin archivo de configuración: todo vive en CSS.

Decisión	Razón, tal como está comentada en el CSS
<code>-primary: oklch(0.56 0.23 27)</code>	El rojo de la pokédex. Es el único color saturado del sistema fuera del arte de las cartas.
<code>-ring: oklch(0.58 0.15 250)</code>	Azul <i>a propósito</i> : «primario y destructivo son ambos rojos, así que un anillo de foco rojo sobre un campo es indistinguible de que ese campo sea inválido».
18 tokens <code>-type-*</code>	Un color por tipo de Pokémon, re-exportados al tema de Tailwind.
<code>TYPE_VAR</code> en TS	Mapea tipo → <i>nombre de variable</i> y no a una clase: «un color de tipo se usa como relleno, resplandor, anillo y parada de degradado, y sólo una variable cruda sirve para las cuatro».
<code>@utility slab / glass / aura / foil</code>	Utilidades propias de Tailwind 4 para la superficie, la cromática flotante, el resplandor teñido por tipo y el holográfico.
<code>forcedTheme="light"</code>	La aplicación es clara y sólo clara: un tema por omisión seguiría al sistema operativo y dejaría a quien lo tenga oscuro en un tema que ya no se mantiene.
<code>prefers-reduced-motion</code>	Todas las animaciones y transiciones se recortan a 0.01 ms.

El paquete no se compila: exporta un archivo por subruta (`@workspace/ui/components/button`) y `apps/web/tsconfig.json` apunta el alias directamente a `packages/ui/src`, así que no existe artefacto intermedio que pueda quedar viejo.

6.8 Empaquetado

`output: "standalone"` traza el grafo de dependencias en un paquete autocontenido, de modo que la imagen de ejecución no lleva `node_modules`. Esa elección obliga a copiar a mano `.next/static` y `public` en la etapa final, y a instalar aparte las dependencias de `apps/web/scripts`, que es un paquete npm propio: Next empaqueta `better-auth` dentro de sus *chunks* de servidor en vez de copiar el paquete, así que la migración no puede tomarlo prestado del árbol trazado.

NEXT_PUBLIC_API_URL es argumento de construcción, no variable de ejecución

Se hornea en el *bundle* del cliente al compilar, así que tiene que ser la URL que alcanza un navegador del anfitrión, no la que usa el contenedor. Cambiarla exige reconstruir la imagen, no reiniciarla. Su contraparte **API_INTERNAL_URL** sí es una variable de ejecución normal, y sólo la lee el servidor en la página pública compartida.

7

Flujos

Tres recorridos completos, elegidos porque cada uno atraviesa una frontera distinta: el modelo de visión, el protocolo MCP y la concurrencia sobre una fila de la base de datos. Los tres diagramas se generan con PlantUML desde `diagramas/src/`.

7.1 Escaneo de una carta

Tres decisiones separadas, y sólo la primera es del modelo:

1. **El modelo lee lo impreso.** El *prompt* de visión dice explícitamente «no infieras ni identifiques de qué carta se trata; otro sistema hace eso». Devuelve nombre, número, total del set y HP.
2. **El código decide qué carta es.** `services/resolve.py` puntúa cada señal (figura 5.2) en lugar de filtrar por ninguna, y clasifica el resultado en `resolved`, `ambiguous` o `failed`.
3. **La persona decide qué se guarda**, y sólo cuando la ambigüedad es real.

Dos detalles operativos: la imagen se guarda *antes* de llamar al modelo, para que un fallo de visión deje algo que reintentar; y si no hay modelo configurado, `create_scan` captura el error y sigue con un `CardReading` vacío, de modo que el escaneo degrada a entrada manual en vez de fallar.

7.2 Un turno del asistente sobre MCP

Lo importante del diagrama es que la aplicación se llama a sí misma por el protocolo público. El chat no consulta `services/` directamente: abre una sesión MCP contra su propio servidor, con *el token del usuario*, y de ahí en adelante es un cliente MCP más. Las consecuencias:

- El agente no puede hacer nada que un asistente externo no pueda hacer. No hay una ruta privilegiada.
- El agente actúa *como el llamante*, no como una identidad de servicio: el aislamiento por usuario lo aplica la misma verificación de token que protege el REST.
- Un turno está acotado: ocho llamadas a herramienta y diez peticiones al modelo. «Un turno que no termina nunca es peor que una respuesta incompleta.»
- El historial se recorta a veinte turnos y se descartan los viejos en lugar de resumirlos, porque las herramientas releen datos vivos en cada turno.

7.3 Tomar una publicación del tablón

Este es el único punto del sistema con competencia real por un recurso: una publicación la puede tomar cualquiera, y sólo una vez. La reserva es una sola sentencia:

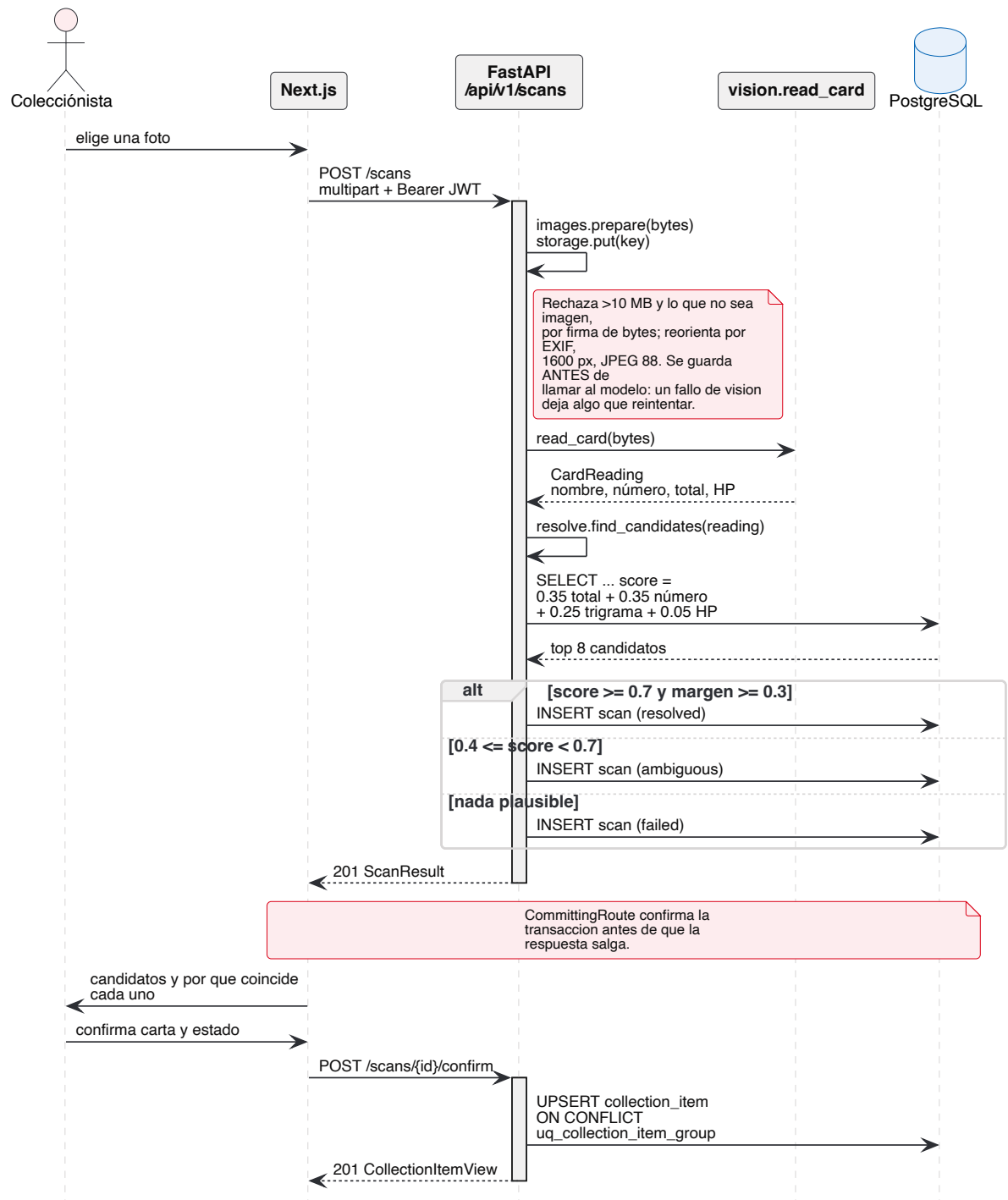


Figura 7.1 POST /api/v1/scans y la confirmación posterior.

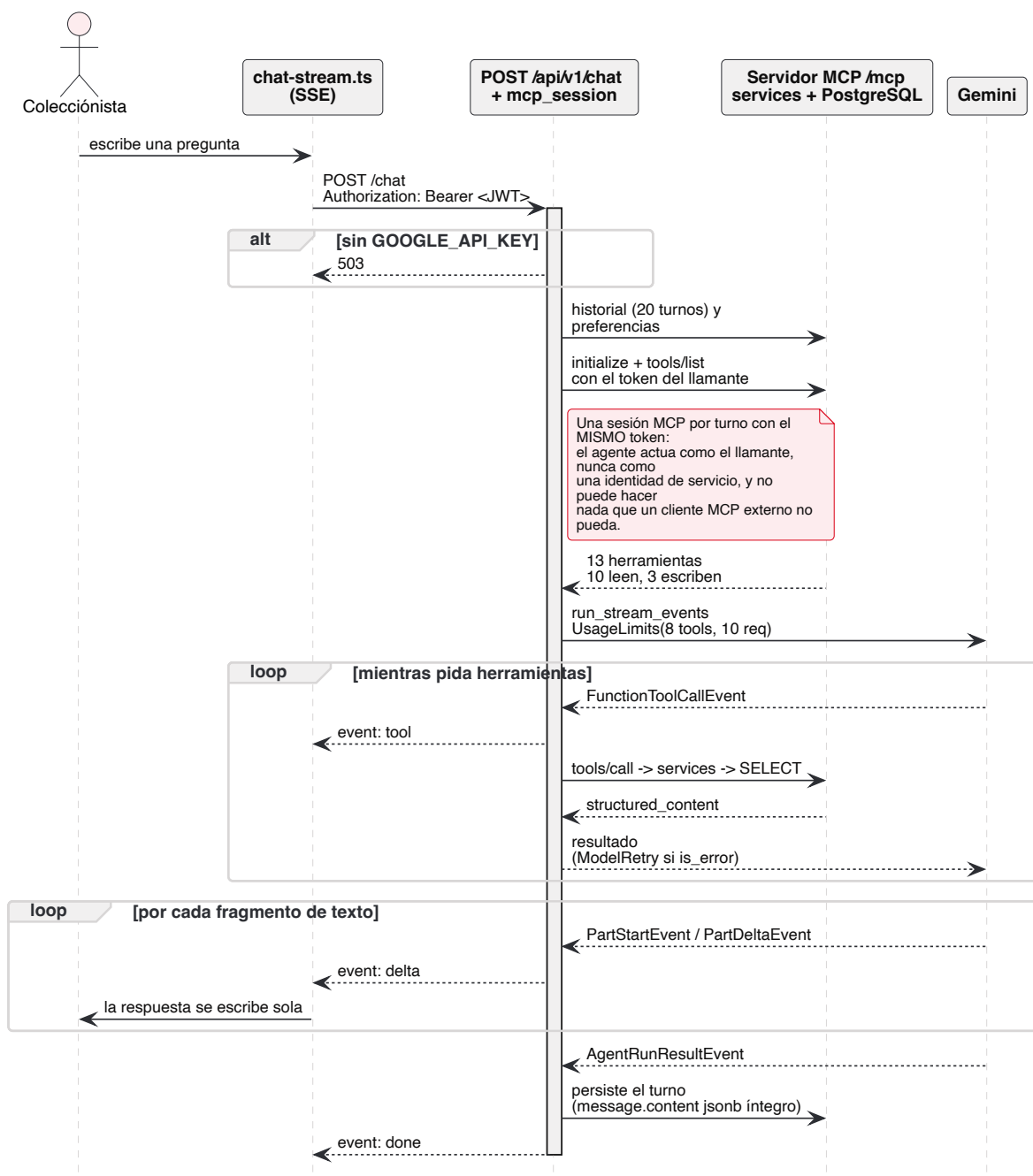


Figura 7.2 `POST /api/v1/chat`: un turno completo, con sesión MCP propia y respuesta en streaming.

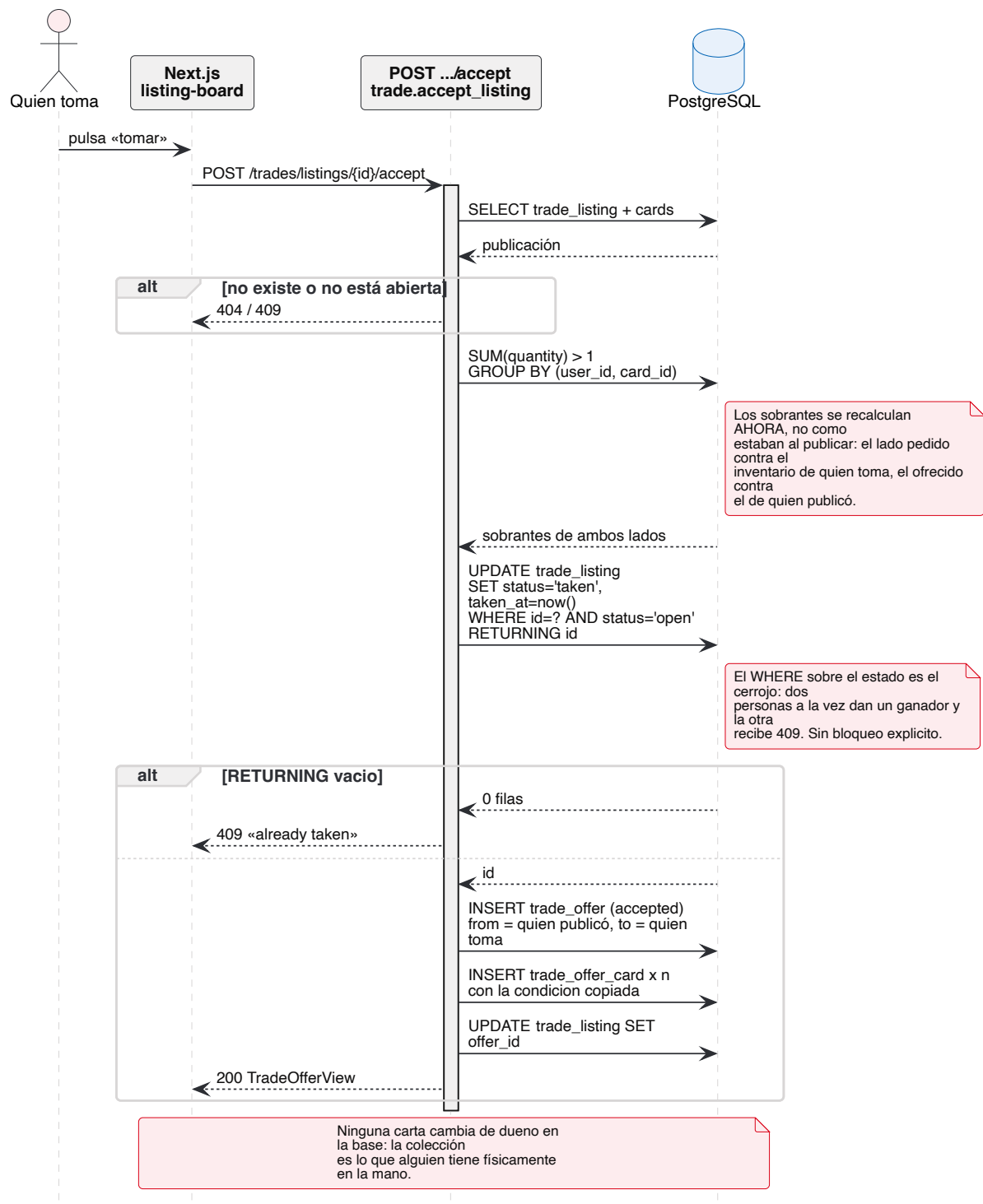


Figura 7.3 `POST /api/v1/trades/listings/{id}/accept`: la reserva atómica y el trato que queda escrito.


```
claimed = (await db.execute(
    update(TradeListing)
    .where(TradeListing.id == listing.id,
           TradeListing.status == ListingStatus.OPEN)
    .values(status=ListingStatus.TAKEN, taken_at=func.now())
    .returning(TradeListing.id)
)).scalar_one_or_none()

if claimed is None:
    raise ListingError("This listing was already taken")
```

La condición sobre el estado dentro del **WHERE** es el cerrojo: dos peticiones simultáneas producen un ganador —el que ve una fila en el **RETURNING**— y la otra recibe 409. No hace falta bloqueo explícito ni un nivel de aislamiento especial.

Antes de la reserva, ambos lados se revalidan contra los sobrantes de *ahora*, no contra los de cuando se publicó. Después, se escribe una **trade_offer** ya aceptada y la publicación queda enlazada a ella, con lo que un trato del tablón y uno negociado son la misma cosa desde ese momento.

Aceptar no mueve cartas

Ni aquí ni al aceptar una oferta cambia una sola fila de **collection_item**. La colección es lo que alguien tiene físicamente, y sólo quien tiene la carta sabe si el intercambio ocurrió. Es exactamente la misma regla que impide al asistente archivar cartas.

8

Calidad

8.1 Estrategia

Dos suites con filosofías distintas, elegidas por lo que cada lado puede equivocarse.

	API (pytest)	Web (Vitest)
Unidad bajo prueba	Un módulo de <code>services/</code>	Un componente o un módulo de <code>lib/</code>
Dependencia real	Postgres de verdad	El cliente HTTP de verdad
Lo que se simula	Sólo las APIs externas (TCGdex, PokeAPI, el modelo)	Sólo la red, y en la frontera de <code>fetch</code>
Aislamiento	Una transacción por prueba, siempre revertida	<code>cleanup()</code> y reinicio de manejadores tras cada prueba
Qué <i>no</i> cubre	El cableado de rutas HTTP	El árbol generado <code>lib/api/**</code>

8.2 Las pruebas de la API

274 funciones de prueba en 23 archivos que, con los cuatro casos parametrizados, pytest recoge como **292 casos**. Todas corren contra un PostgreSQL real; ninguna deja una fila detrás.

```
@pytest.fixture
async def db() -> AsyncIterator[AsyncSession]:
    """Session bound to an open transaction that is always rolled back.

    Every test sees a real Postgres - enums, check constraints and ON CONFLICT
    behave as in production - while leaving no rows behind.
    """
    engine = create_engine()
    connection = await engine.connect()
    transaction = await connection.begin()
    session = AsyncSession(bind=connection, expire_on_commit=False)
    try:
        yield session
    finally:
        await session.close()
        await transaction.rollback()
        await connection.close()
        await engine.dispose()
```

La sesión se ata a una conexión con una transacción ya abierta *desde fuera*. Cualquier `session.commit()` dentro del código bajo prueba confirma sólo la unidad interna; el `rollback` del `finally` descarta todo. No hay truncados ni recreación de esquema entre pruebas, así que son independientes del orden y rápidas —los 292 casos tardan menos de catorce segundos.

La suite es de capa de servicio, no de ruta

No hay `TestClient` ni transporte ASGI en ninguna prueba: se ejercita `services/`, no `api/v1/`. El cableado de rutas—incluido `CommittingRoute`, que es justo la pieza menos evidente del backend—no está cubierto directamente. Quien toque esa capa debería verificarla a mano o añadir la prueba que hoy falta.

Lo que se gana con ello es exactamente lo que un doble de prueba no puede dar: los enums de Postgres, los `CHECK`, el `ON CONFLICT ... NULLS NOT DISTINCT` y la semántica de `RETURNING` se comportan como en producción. Los usuarios de prueba se insertan en `auth."user"` con SQL crudo, porque esa tabla es de Better Auth y duplicar su modelo sería inventar una segunda fuente de verdad.

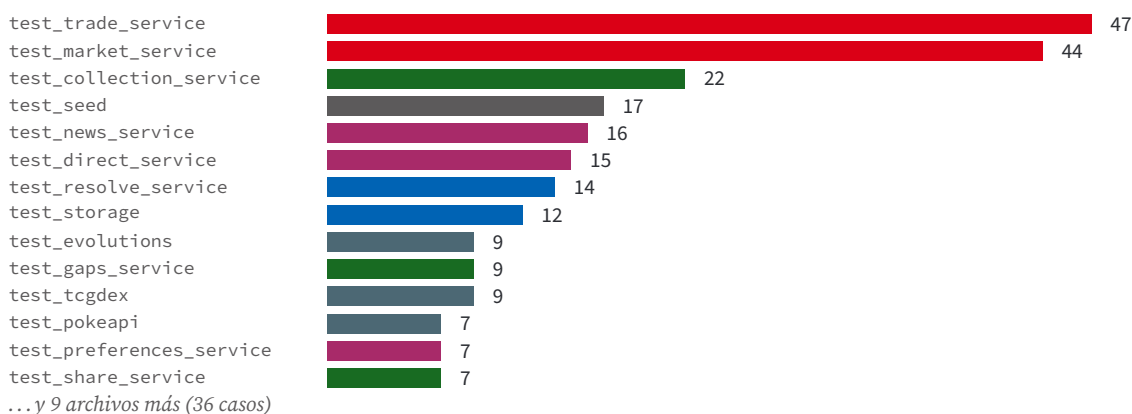


Figura 8.1 Dónde está el esfuerzo de prueba en la API: intercambio y mercado son la mitad de la suite, que es también donde está la aritmética delicada.

8.3 Las pruebas de la web

71 casos en 7 archivos. La decisión de diseño está escrita en `apps/web/test/server.ts`:

La red se simula en la frontera de `fetch` en vez de sustituyendo el cliente generado, para que `apiClient`—la capa que convierte un cuerpo 4xx en un `ApiError`—se ejecute de verdad en cada prueba de componente.

Sustituir el cliente generado dejaría sin probar precisamente el código propio: el orden de las cabeceras, la caché del token, la traducción de errores de FastAPI. Con MSW, un componente que renderiza un estado de error lo hace recorriendo la misma ruta que en producción.

El servidor MSW arranca con `onUnhandledRequest: "error"`: cualquier petición que una prueba no haya declarado *falla* la prueba, en lugar de colarse a la red. El único manejador global es `GET */api/auth/token`, que devuelve `"test-token"`; todo lo demás se declara por prueba.

Archivo	Casos	Qué asegura
lib/format.test.ts	16	Plurales, moneda en formato en-US, cuotas, «ahora»/«Hoy»/«Ayer» contados por día natural y no por horas transcurridas.
lib/api-client.test.ts	12	304 pasa sin lanzar; detail en sus dos formas; el token se acuña una sola vez; un 401 al acuñar deja la petición anónima; FormData no lleva Content-Type ; una cabecera del llamante no puede desplazar al token.
lib/url-state.test.tsx	9	Los valores por defecto sólo rellenan huecos; se escribe con replaceState y no pushState ; "" y undefined borran; popstate relea; el escucha se retira.
components/card-image.test.tsx	10	El recorte a ventana apaisada sea cual sea la forma del origen, la proporción impresa 63×88, el escaneo bloqueado en gris.
components/dialog-toolbar.test.tsx	9	Búsqueda, marcado del filtro activo y paginación que se oculta cuando todo cabe.
components/wish-button.test.tsx	8	Se quita por el id de la <i>entrada de deseo</i> , no por el de la carta; stopPropagation para que el toque no abra la carta detrás; deshabilitado mientras la primera pulsación está en vuelo.
components/portfolio-simulator.test.tsx	7	Quién sale ganando, «swap parejo» por debajo de un dólar y el aviso cuando el valor se concentra en una carta.

Tres piezas de infraestructura merecen mención porque explican fallos que, de otro modo, cuestan horas: un **Environment** propio de Vitest restaura el **AbortController** de Node sobre el de jsdom —Node **fetch** rechaza una señal ajena, y React Query pasa una señal a cada consulta—; **next/image** se sustituye por un doble que conserva **fill** y **priority** como atributos de datos; y **esbuild.js**: **"automatic"** está puesto porque el **tsconfig** de la app deja la transformación de JSX en manos de Next, que aquí no interviene.

8.4 Análisis estático

Herramienta	Ámbito	Configuración
mypy	apps/api/src	strict = true , Python 3.13, ignore_missing_imports
ruff	apps/api	select = ["E", "F", "I", "N", "UP", "B", "SIM", "RUF"] , línea de 100, excluye alembic/versions porque esos archivos los escribe la plantilla y se revisan a mano
tsc -noEmit	apps/web, packages/ui	strict, noUncheckedIndexedAccess, isolatedModules
eslint	Todo el workspace	Base + Next + React Hooks + turbo/no-undeclared-env-vars

El lint del frontend nunca falla

packages/eslint-config/base.js carga **eslint-plugin-only-warn**, que degrada *toda* regla a advertencia. La ejecución que acompaña a este documento reporta 116 advertencias y 0 errores; el job de CI pasa igualmente. Es una elección deliberada de ergonomía, pero significa que el lint es informativo y no una puerta.

8.5 Ejecución de referencia

Lo siguiente es la salida real obtenida al preparar este manual, en la revisión [c5d3bd3](#), contra el mismo Postgres 18 que usa el entorno local.

```
$ cd apps/api && uv run pytest -q
..... [ 98%]
.... [100%]
292 passed in 13.61s

$ uv run mypy src
Success: no issues found in 87 source files

$ uv run ruff check .
All checks passed!

$ npx turbo run test --filter=web...
RUN v3.2.7 /Users/.../apps/web
v lib/url-state.test.tsx (9 tests) 21ms
v lib/format.test.ts (16 tests) 8ms
v lib/api-client.test.ts (12 tests) 37ms
v components/card-image.test.tsx (10 tests) 73ms
v components/wish-button.test.tsx (8 tests) 133ms
v components/dialog-toolbar.test.tsx (9 tests) 158ms
v components/portfolio-simulator.test.tsx (7 tests) 475ms

Test Files 7 passed (7)
Tests 71 passed (71)
Duration 1.85s

$ npx turbo run typecheck lint --filter=web...
x 116 problems (0 errors, 116 warnings)
Tasks: 4 successful, 4 total
```

Medida	Valor	Fuente
Casos de prueba en la API	292	pytest -q
Funciones de prueba en la API	274	23 archivos, 4 parametrizados
Duración de la suite de la API	13.61 s	pytest -q
Casos de prueba en la web	71	vitest run
Duración de la suite web	1.85 s	vitest run
Archivos bajo mypy -strict	87	mypy src
Errores de lint	0	ruff,eslint
Advertencias de lint (web)	116	eslint

9

Integración continua y despliegue

9.1 El pipeline

Un solo flujo, `.github/workflows/ci.yml`, disparado en `push` y en `pull_request`, con `concurrency` por rama y `cancel-in-progress`: un empujón nuevo cancela la ejecución anterior de la misma rama. Dos jobs independientes que corren en paralelo.

9.1.1 Job API

Levanta un *service container* `postgres:18-alpine` con `pg_isready` como healthcheck, y después ejecuta una secuencia que reproduce el arranque real del sistema. Tres pasos existen por razones que no son obvias y están comentadas en el propio flujo:

- **Crear los esquemas a mano.** Un service container no monta `docker-entrypoint-initdb.d`, así que el paso ejecuta `db/init/01-schemas.sql` —los dos esquemas y la extensión `pg_trgm`— tal como lo haría el fichero de compose.
- **Migrar el esquema `auth` antes que el de dominio.** Las tablas de dominio tienen claves foráneas hacia `auth."user"`; sin este paso la primera migración de Alembic no puede correr.
- **Sembrar un set del catálogo.** Las pruebas de servicio buscan cartas reales en vez de inventarlas, así que el catálogo tiene que estar. Un set basta y tarda unos ocho segundos.

Después, `pytest -q, mypy src` y `ruff check ..`

CI corre sin ninguna clave de modelo

`GOOGLE_API_KEY` se deja explícitamente vacía, con este comentario en el flujo: «el agente es opcional, el chat responde 503 sin él, y un repositorio público no debe necesitar una clave para ponerse en verde». Es la prueba ejecutable de **RNF-07**.

9.1.2 Job Web

Tres pasos: `checkout`, `npm ci` y `npx turbo run typecheck lint test -filter=web...`. El filtro acota la ejecución al workspace de la web y a sus dependencias; las tareas de raíz cubrirían también la API, cuyo utillaje vive en el otro job.

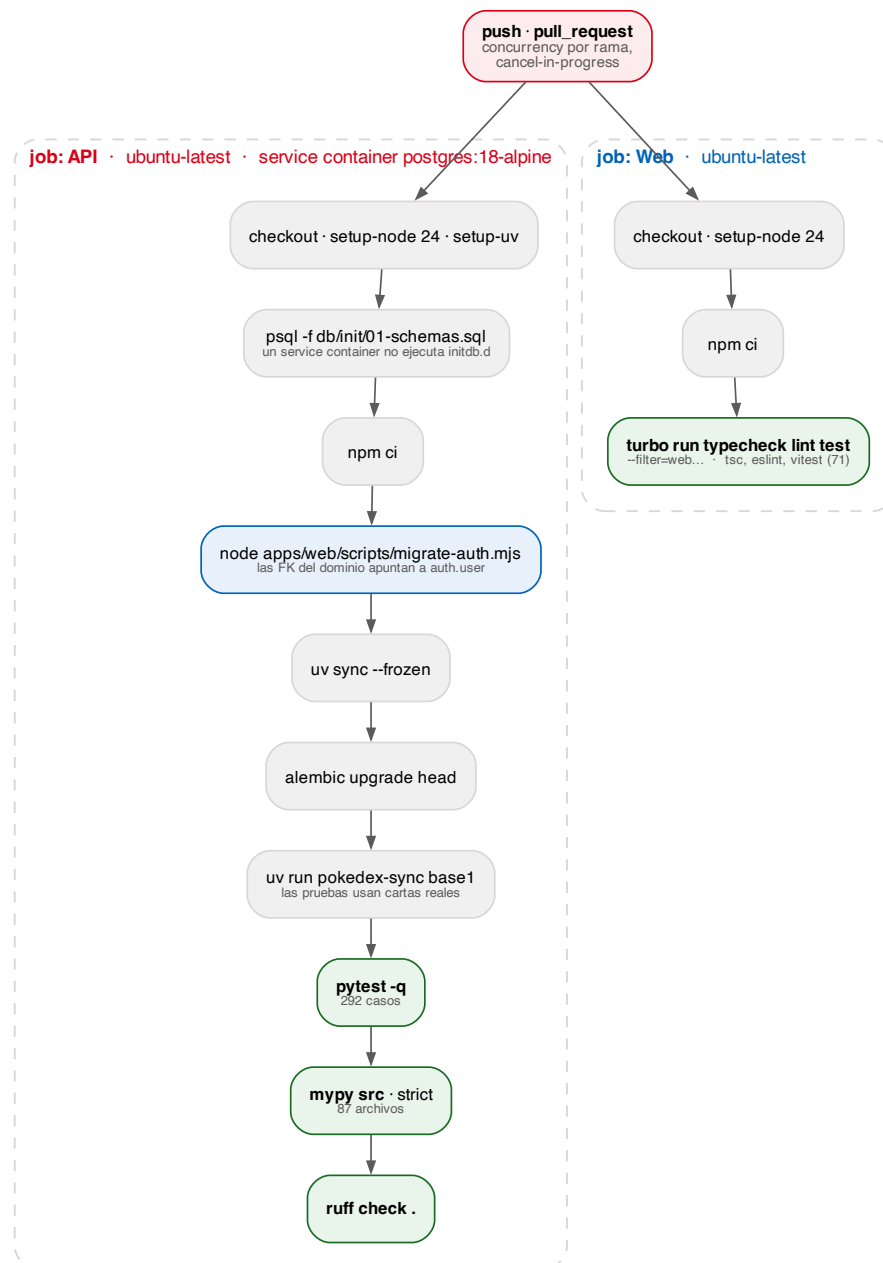


Figura 9.1 Los dos jobs y sus pasos, en orden. Generado desde `diagramas/src/ci-pipeline.dot`.

9.2 Imágenes

Servicio	Base	Notas
api	<code>uv:python3.13-bookworm-slim</code>	Capa de dependencias sólo con el <i>lockfile</i> (<code>uv sync --frozen --no-install-project --no-dev</code>), luego el código. <code>UV_COMPILE_BYTECODE=1</code> .
web	<code>node:22-slim</code> , dos etapas	Etapas de construcción con <code>npm ci</code> sobre los <code>package.json</code> antes de copiar el código; etapa final que ensambla la traza <code>standalone</code> .

El arranque de cada imagen levanta el esquema que le pertenece. La web ejecuta `node apps/web/scripts/migrate-auth.mjs` && `node apps/web/server.js`. La API espera:

```
# The domain tables carry foreign keys into 'auth.user', which Better Auth
# creates from the web service. Both start at once on a fresh deployment, so
# this waits for that schema rather than crash-looping.
for i in $(seq 1 30); do alembic upgrade head && break; \
echo 'waiting for the auth schema'; sleep 5; done; \
alembic current | grep -q . && exec fastapi run src/pokedex/main.py \
--host 0.0.0.0 --port 8000
```

La configuración de migración está duplicada a propósito

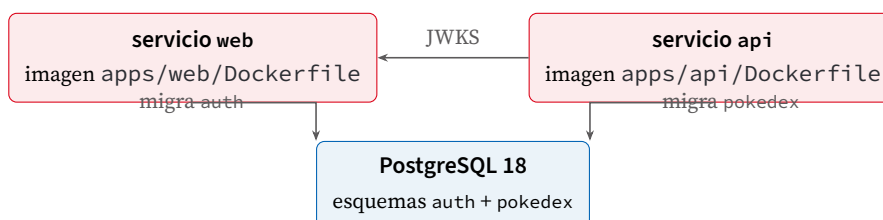
`apps/web/scripts/migrate-auth.mjs` repite la configuración de `lib/auth.ts` porque no puede importarla: la construcción `standalone` entrega *bundles*, no el TypeScript fuente. El propio archivo lo advierte: «un plugin añadido allí y olvidado aquí es una tabla que esto nunca crea». Cualquier cambio en la configuración de Better Auth hay que hacerlo en los dos sitios.

9.3 Forma del despliegue

Producción corre en **Railway**, con dos servicios y una base de datos:

Servicio	URL pública
web	<code>https://web-production-78a4b.up.railway.app</code>
api	<code>https://api-production-81ae.up.railway.app</code>
db	PostgreSQL 18 gestionado, sólo accesible desde la red privada

No hay configuración de proveedor versionada en el repositorio: ni `railway.json`, ni `nixpacks.toml`, ni `Procfile`. Lo que está definido son las dos imágenes y el contrato de variables, y eso basta: el sistema se despliega igual en cualquier anfitrión que ejecute contenedores —Railway, Fly, Render o un Kubernetes— con la misma topología:



Servicio	Variables que el anfitrión debe fijar
api	<code>DATABASE_URL</code> (forma <code>asyncpg</code>), <code>AUTH_JWKS_URL</code> , <code>AUTH_ISSUER</code> , <code>AUTH_AUDIENCE</code> , <code>CORS_ORIGINS</code> , <code>PORT</code>
web	<code>DATABASE_URL</code> (forma <code>postgresql://</code> , es <code>node-pg</code>), <code>BETTER_AUTH_URL</code> , <code>BETTER_AUTH_SECRET</code> , <code>NEXT_PUBLIC_API_URL</code> , <code>PORT</code> , <code>HOSTNAME=0.0.0.0</code>

Cuatro puntos que deciden si el despliegue funciona a la primera:

1. **NEXT_PUBLIC_API_URL es un argumento de construcción.** Next lo hornea en el bundle del cliente, así que tiene que ser la URL *pública* de la API, la que resuelve un navegador —no la interna del contenedor. Cambiarla obliga a reconstruir la imagen. Si el anfitrión sólo permite variables de ejecución, hay que pasarla como *build arg* del contenedor.
2. **El emisor y el JWKS son URLs distintas.** `AUTH_ISSUER` es lo que el token declara y lo que ve el navegador; `AUTH_JWKS_URL` es por dónde el contenedor de la API alcanza al de la web. En una red interna serán diferentes, y eso es correcto.
3. **Los dos DSN tienen formas distintas.** La API usa `postgresql+asyncpg://`; la web usa `postgresql://` porque su cliente es `node-pg`.
4. **CORS_ORIGINS tiene que incluir el origen público de la web**, porque el navegador llama a la API cruzando origen.

Para poblar un despliegue nuevo:

```
pokedex-sync base1 base2 base3      # catalogo
pokedex-seed --count 1000           # un mercado de coleccionistas
pokedex-demo                         # tres cuentas de demostracion
```

10

Manual de instalación

Los pasos de este capítulo se ejecutaron desde cero, sobre un clon limpio del repositorio y una base de datos vacía, mientras se escribía este documento. Las salidas que se citan son las que produjeron.

10.1 Requisitos previos

Herramienta	Versión	Para qué
Docker	cualquiera reciente	PostgreSQL 18 (y, opcionalmente, toda la pila)
Node.js	≥ 20	Frontend y Better Auth; CI usa 24
npm	11.x	Gestor de workspaces del monorepo
uv	reciente	Entorno y dependencias de Python
Python	3.13	Lo instala uv si no está

Opcional: una clave de Google Gemini para el chat y el escaneo, o bien Ollama para ejecutar visión en local.

10.2 Puesta en marcha, paso a paso

10.2.1 1 · Clonar y preparar el entorno

```
git clone https://github.com/Skulltulaidm/pokedex-manager.git
cd pokedex-manager      # la rama por omision es main

cp .env.example .env
cp apps/api/.env.example apps/api/.env
```

El `.env` de la raíz lo leen Docker Compose y la web; el de `apps/api` lo lee la API cuando se ejecuta fuera de contenedor. Si el puerto 5432 ya está ocupado, cambie `POSTGRES_PORT` en el primero y el puerto de `DATABASE_URL` en el segundo: tienen que coincidir.

10.2.2 2 · Levantar la base de datos

```
docker compose up -d db
```

El contenedor se publica *sólo* en `127.0.0.1`, porque las credenciales por omisión son débiles. Al inicializarse ejecuta `db/init/01-schemas.sql`, que crea los dos esquemas y la extensión de trigramas. Comprobación:

```
$ docker compose exec db psql -U pokedex -d pokedex -c "\dn"
```

Name	Owner
auth	pokedex
pokedex	pokedex
public	pg_database_owner

Si el volumen ya existía

db/init sólo se ejecuta la primera vez, con el volumen vacío. Si la base ya existía sin esquemas, créelos a mano:

```
docker compose exec db psql -U pokedex -d pokedex \
  -f /docker-entrypoint-initdb.d/01-schemas.sql
```

10.2.3 3 · Instalar dependencias de JavaScript

```
npm install
```

10.2.4 4 · Migrar el esquema **auth**

Este paso va antes que el siguiente, siempre: las tablas de dominio tienen claves foráneas hacia **auth."user"**.

```
npx @better-auth/cli@latest migrate -c apps/web --yes
```

Crea cinco tablas —**user**, **session**, **account**, **verification** y **jwt**— y termina con **migration was completed successfully!**. La alternativa, y es la que usan la imagen de Docker y CI, es el script del repositorio:

```
DATABASE_URL=postgresql://pokedex:pokedex@localhost:5432/pokedex \
  node apps/web/scripts/migrate-auth.mjs
```

10.2.5 5 · Migrar el esquema **pokedex** y sembrar el catálogo

```
cd apps/api
uv sync
uv run alembic upgrade head
uv run pokedex-sync base1
cd ../../
```

Las catorce revisiones dejan la base en **d7a41f0c92be** (**head**). La sincronización de **base1** tardó **6.9 s** y reportó **base1: 102 cards, 69 species**. Para más sets, añade identificadores: **pokedex-sync base1 base2 base3**. La lista está en <https://api.tcgdex.net/v2/en/sets>.

10.2.6 6 · Arrancar

```
npm run dev # web en :3000, api en :8010
```

Abra **http://localhost:3000** y cree una cuenta.

10.2.7 7 · El modelo (opcional)

```
# en .env de la raíz
GOOGLE_API_KEY=...
```

Sin clave, todo funciona salvo dos cosas: el chat responde 503 y el escaneo degrada a entrada

manual. Para ejecutar visión en local:

```
ollama pull qwen2.5vl:7b
echo 'VISION_MODEL=ollama:qwen2.5vl:7b' >> .env
```

El contenedor de la API alcanza un Ollama del anfitrión por `host.docker.internal`, que ya está cableado en el fichero de compose.

10.3 Verificación de la instalación

```
$ curl -s http://localhost:8010/health
{"status":"ok"}

$ curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8010/openapi.json
200

$ curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8010/api/v1/collection
401
```

El 401 sin token es la comprobación importante: significa que la verificación de JWT está activa. La referencia interactiva de la API está en <http://localhost:8010/scalar>.

Suites completas:

```
npm run test          # 292 casos de API + 71 de web
npm run typecheck     # mypy --strict + tsc
npm run lint
```

10.4 Datos de ejemplo

Comando	Qué hace
<code>pokedex-sync base1 base2 base3</code>	Sincroniza sets del catálogo desde TCGdex y PokeAPI.
<code>pokedex-seed -count 1000</code>	Escribe un mercado de coleccionistas reproducible: colecciones, deseos, ofertas y publicaciones. Todo descende de una sola semilla (<code>-seed</code> , por omisión <code>20260813</code>), así que repetirlo no escribe nada nuevo. <code>-reset</code> borra sólo lo sembrado, identificado por el prefijo <code>seed-</code> .
<code>pokedex-demo</code>	Da de alta tres cuentas de demostración por HTTP —una por cada pregunta que responde la aplicación— usando el propio servicio web para el registro.

10.5 Todo en contenedores

```
# BETTER_AUTH_SECRET es obligatorio; compose falla rapido sin el
docker compose up -d --build
```

Las tres imágenes se orquestan solas: la web migra `auth` antes de servir, y la API reintenta `alembic upgrade head` hasta treinta veces esperando ese esquema. Recuerde que `NEXT_PUBLIC_API_URL` se pasa como argumento de construcción (`http://localhost:${API_PORT}`), así que cambiar el puerto publicado obliga a reconstruir la imagen de la web.

10.6 Problemas frecuentes

Síntoma	Causa y remedio
<code>relation "auth.user" does not exist</code> al migrar	Se saltó el paso 4. Ejecute la migración de Better Auth y repita <code>alembic upgrade head</code> .
<code>schema "pokedex" does not exist</code>	El volumen de Postgres ya existía cuando se añadió <code>db/init</code> . Ejecute el SQL a mano (véase el aviso del paso 2).
<code>operator does not exist: text% text</code>	Falta la extensión <code>pg_trgm</code> en <code>public</code> . <code>CREATE EXTENSION IF NOT EXISTS pg_trgm SCHEMA public;</code>
El chat responde 503	No hay <code>GOOGLE_API_KEY</code> . Es el comportamiento esperado, no un fallo.
El navegador da error de CORS	<code>CORS_ORIGINS</code> no incluye el origen desde el que se abre la aplicación.
Peticiones a la API sin token	Falta <code>{ client: { client: apiClient } }</code> en el sitio de llamada del hook generado.
Puerto 5432 ocupado	Ajuste <code>POSTGRES_PORT</code> en <code>.env</code> y el puerto de <code>DATABASE_URL</code> en <code>apps/api/.env</code> .
Kubb no genera nada	La API tiene que estar sirviendo: la entrada por omisión es <code>http://localhost:8010/openapi.json</code> .

11

Bitácora de decisiones

Las decisiones que un lector nuevo cuestionaría, con la alternativa que se descartó y por qué. La mayoría están argumentadas en los mensajes de commit y en los comentarios del propio código, que es de donde salen.

Tabla 11.1. Decisiones de arquitectura

ID	Decisión	Alternativa descartada	Razón
AD-01	Dos esquemas en una base, migrados por herramientas distintas.	Un solo esquema y un solo migrador.	Cada esquema lo define el sistema que sabe cómo debe ser. Better Auth crea la tabla que un plugin nuevo necesita; Alembic no puede saberlo. El riesgo (que autogenerate proponga borrar lo ajeno) se cierra con dos filtros.
AD-02	Carta y especie son capas separadas, unidas por el <code>dexId</code> del origen.	Una tabla con el nombre del Pokémon dentro de la carta.	Una carta es una impresión y una especie es lo retratado; unir las por nombre falla con formas regionales, cartas de entrenador y traducciones.
AD-03	El escaneo puntúa todas las señales en lugar de filtrar por alguna.	Filtrar por número de colector o por set.	Un campo mal leído descartaría la carta correcta. Puntuando, un HP erróneo cuesta 0.05 y la respuesta sigue apareciendo.
AD-04	<code>CommittingRoute</code> confirma antes de que la respuesta salga.	Confiar en el commit de la dependencia <code>yield</code> .	Ese commit ocurre <i>después</i> de responder, así que un refetch inmediato puede leer la base antes que él y devolver el estado viejo. Con <code>staleTime: 0</code> en el cliente, esa carrera es la norma y no la excepción.
AD-05	El chat de la aplicación es un cliente MCP más.	Que el endpoint de chat llame a <code>services/</code> directamente.	Una sola superficie de datos y un solo modelo de permisos. El agente no puede hacer nada que un asistente externo no pueda, y actúa con el token del usuario, no como servicio.
AD-06	Aceptar un intercambio registra el acuerdo pero no mueve cartas.	Transferir filas de <code>collection_item</code> .	La colección es lo que alguien tiene físicamente; sólo esa persona sabe si el intercambio ocurrió. Mover filas convertiría un acuerdo en una mentira sobre el mundo.
AD-07	Un sobrante se calcula (<code>SUM(quantity) > 1</code>), no se marca.	Una columna «disponible para cambio».	Un estado marcado se desincroniza de las cantidades reales. Calculado, el tablón nunca puede prometer una carta que ya no existe, y se revalida al publicar, al ofertar y al aceptar.
AD-08	<code>card_price</code> guarda una fila por carta y por día.	Guardar sólo el último precio en <code>card</code> .	«Un portafolio sin serie es un número suelto»: todo panel de variación reportaba +0.0 % sobre un único punto. El upsert sobre (<code>card_id</code> , <code>recorded_on</code>) mantiene un punto por día y hace el trabajo idempotente.

Continúa en la página siguiente

Tabla 11.1. Decisiones de arquitectura *(continuación)*

ID	Decisión	Alternativa descartada	Razón
AD-09	La lectura de precios es una tarea suelta al arrancar, no un planificador.	Cron, o un worker aparte.	«Un catálogo inalcanzable es una serie desactualizada, no una aplicación que no arranca.» La deduplicación por día de calendario hace irrelevante cuántas veces se reinicie el servicio.
AD-10	Una publicación es una oferta sin destinatario; tomarla escribe una oferta ya aceptada.	Un modelo de subasta o de reserva.	Tomarla es el acuerdo. La reserva atómica (<code>UPDATE ... WHERE status='open' ... RETURNING</code>) resuelve la concurrencia sin bloqueos, y desde ahí un trato del tablón es idéntico a uno negociado.
AD-11	La condición se copia sobre la carta de la oferta.	Referenciar la fila de colección de origen.	Una oferta es una promesa sobre una carta en un estado, y debe sobrevivir a que su dueño edite o borre esa fila.
AD-12	Un hilo directo es un par ordenado con <code>CHECK</code> .	Emisor y destinatario, o una tabla de participantes.	Con un par no ordenado, dos personas escribiéndose a la vez abren dos hilos y cada lectura tiene que unirlos. No hay tabla de participantes porque no hay un tercero.
AD-13	Los filtros escriben la URL con la History API.	<code>router.replace</code> .	En Next eso es una navegación aunque sólo cambie un parámetro: refetch del payload de ruta y remontado del árbol. Cada clic en un filtro parpadeaba con un esqueleto y volvía a pedir lo que ya estaba en pantalla. El precio —que <code>useSearchParams</code> deja de enterarse— se paga concentrando la lectura en un solo hook.
AD-14	El cliente HTTP se genera del OpenAPI y se versiona en git.	Escribirlo a mano, o generarlo en tiempo de construcción.	El contrato lo fija el servidor; el frontend no puede desviarse en silencio. Versionarlo hace que un cambio de API aparezca como diff revisable. Su envoltorio se inyecta por llamada porque es lo que Kubbb permite.
AD-15	Las pruebas de la API corren contra Postgres real con rollback por prueba.	SQLite en memoria, o dobles del repositorio.	La mitad de las invariantes viven en la base: enums, <code>CHECK</code> , <code>ON CONFLICT ... NULLS NOT DISTINCT</code> , <code>RETURNING</code> . Un doble las daría todas por buenas. Y son rápidas igualmente: 292 casos en 13.6 s.
AD-16	Las pruebas de la web simulan la red en la frontera de <code>fetch</code> .	Sustituir el cliente generado.	Sustituirlo dejaría sin probar el único código propio de esa capa: el orden de cabeceras, la caché del token y la traducción de errores de FastAPI.
AD-17	El <code>toolset</code> MCP del agente está escrito a mano.	Usar el cliente MCP de pydantic-ai.	Ese cliente apunta al SDK 1.x y ni siquiera importa contra 2.0, que renombró los campos del protocolo y movió <code>RequestContext</code> .
AD-18	Las imágenes de escaneo van a un volumen local tras un <code>Protocol</code> .	MinIO o S3.	«Añadiría un cuarto contenedor a algo que sólo corre en local.» La costura está puesta: cambiar de backend es implementar tres métodos.
AD-19	<code>operationId</code> es el nombre de la función manejadora.	El identificador por omisión de FastAPI.	El de fábrica produce ganchos como <code>useListCollectionApiV1CollectionGet</code> .

Continúa en la página siguiente

Tabla 11.1. Decisiones de arquitectura *(continuación)*

ID	Decisión	Alternativa descartada	Razón
AD-20	Un solo tema, claro, forzado.	Tema oscuro, o seguir al sistema operativo.	Se retiró el tema oscuro a petición del usuario, y <i>forzarlo</i> en vez de poner un valor por omisión: un valor por omisión sigue obedeciendo al sistema operativo y dejaría a una máquina en oscuro dentro de un tema que la aplicación ya no tiene. Costó un bloque de variables porque la paleta estaba escrita como tokens semánticos —superficies y bordes, nunca un gris literal.

Una nota de mantenimiento

El bloque `:root` de `packages/ui/src/styles/globals.css` conserva un comentario de la etapa anterior («dark is the designed state») que los valores de debajo ya contradicen. No afecta a nada en ejecución, pero conviene saberlo antes de leer ese archivo buscando la intención de diseño: la intención vigente es la de AD-20.

A

Anexos

A.1 Variables de entorno, completas

Tabla A.1. Variables de entorno

Variable	Ámbito	Por omisión	Notas
POSTGRES_USER	compose	pokedex	—
POSTGRES_PASSWORD	compose	pokedex	Débil a propósito; por eso el puerto sólo se publica en 127.0.0.1.
POSTGRES_DB	compose	pokedex	—
POSTGRES_PORT	compose	5432	Puerto del anfitrión.
API_PORT / WEB_PORT	compose	8010 / 3000	Dentro del contenedor la API siempre escucha en 8000.
BETTER_AUTH_URL	web	http://localhost:3000	Origen público de la web.
BETTER_AUTH_SECRET	web	obligatoria	Compose falla rápido si falta.
NEXT_PUBLIC_API_URL	web	http://localhost:8010	Argumento de construcción. URL pública de la API.
API_INTERNAL_URL	web	http://localhost:8010	Variable de ejecución; sólo la lee la página pública compartida.
DATABASE_URL	web	obligatoria	Forma postgresql:// (node-pg), con search_path=auth.
DATABASE_URL	api	obligatoria	Forma postgresql+asyncpg://.
CORS_ORIGINS	api	http://localhost:3000	Lista separada por comas.
AUTH_JWKS_URL	api	obligatoria	Cómo alcanza la API el JWKS.
AUTH_ISSUER	api	obligatoria	Lo que el token declara.
AUTH_AUDIENCE	api	obligatoria	—
MCP_RESOURCE_URL	api	http://localhost:8010/mcp	URL que se anuncia al descubrimiento externo.
MCP_INTERNAL_URL	api	vacía	Por dónde se conecta el agente interno; en contenedor, el puerto publicado no existe.
STORAGE_ROOT	api	./data/scans	En Docker, /data/scans.

Continúa en la página siguiente

Tabla A.1. Variables de entorno (continuación)

Variable	Ámbito	Por omisión	Notas
PRICE_REFRESH_ON_START	api	true	Ponerla a falso en pruebas y en ejecuciones sin red.
AGENT_MODEL	api	google-gla: gemini-flash-latest	Alias, no versión fijada.
VISION_MODEL	api	google-gla: gemini-flash-latest	Prefijo ollama: para un modelo local.
GOOGLE_API_KEY	api	vacía	Credencial e interruptor del agente.
OLLAMA_URL	api	http://localhost: 11434/v1	En contenedor, http://host. docker.internal:11434/v1.

A.2 Comandos de referencia

Comando	Qué hace
npm run dev	Web en :3000 y API en :8010, en paralelo.
npm run test	Ambas suites vía Turborepo.
npm run typecheck	mypy -strict y tsc -noEmit.
npm run lint	ruff y eslint.
uv run alembic upgrade head	Aplica las migraciones de dominio.
uv run alembic revision -autogenerate -m "..."	Nueva revisión, filtrada al esquema pokedex.
uv run pokedex-sync <set-id>...	Sincroniza sets del catálogo.
uv run pokedex-seed -count N	Mercado reproducible. -reset borra sólo lo sembrado.
uv run pokedex-demo	Tres cuentas de demostración por HTTP.
npx kubb generate	Regenera apps/web/lib/api (la API debe estar sirviendo).
node apps/web/scripts/migrate-auth.mjs	Migra el esquema auth sin el CLI.

A.3 Conectar un asistente externo por MCP

Con la sesión iniciada, GET /api/auth/token devuelve un JWT. Apunte cualquier cliente MCP al servidor:

```
{
  "mcpServers": {
    "pokedex": {
      "url": "http://localhost:8010/mcp",
      "headers": { "Authorization": "Bearer <token>" }
    }
  }
}
```

Diez herramientas leen, tres escriben, y ninguna de las tres alcanza la colección (capítulo 5).

A.4 Cómo se construye este documento

```
cd docs/manual-tecnico
make          # genera las figuras y despues el PDF
make diagramas # solo las figuras
make limpiar  # borra todo lo generado
```

Requisitos: `pdflatex` y `latexmk` de TeX Live, `plantuml` con un JRE, `graphviz` y `rsvg-convert` de librsvg. Ninguna figura es una captura de pantalla: las nueve se generan desde el código fuente que se versiona en `diagramas/src/`, y el resto son TikZ dentro del propio documento.

Fuente	Herramienta	Figura
c4-contexto.dot	Graphviz	3.1
er-catalogo.dot	Graphviz	4.1
er-coleccion.dot	Graphviz	4.2
er-intercambio.dot	Graphviz	4.3
er-mensajeria.dot	Graphviz	4.4
ci-pipeline.dot	Graphviz	9.1
seq-escaneo.puml	PlantUML	7.1
seq-chat-mcp.puml	PlantUML	7.2
seq-tablon.puml	PlantUML	7.3
Inline	TikZ	3.2, 5.1, 5.2, 8.1